

ELEC374 – Final Report

Group - 5

April 4th, 2024

Luke Hunker (20287422)

Rhys Blakeney (20283269)

Ajmal Niazi (20300697)

Abstract

The CPU design project aims to create a simple RISC-style computer known as a Mini SRC, featuring a 32-bit architecture with essential components such as the RISC processor, memory subsystem, and I/O functionality. The design was made using the Intel Quartus Prime software and simulated using Intel ModelSim, with implementation targeted at a Cyclone V chip with the DEO-CV development board.

The project divides into four primary phases: Phase 1 focuses on register and ALU design, Phase 2 implements key datapath modules such as Select Encode and CONFF, Phase 3 involves control unit development and simulation, and Phase 4 concludes the project with FPGA hardware simulation.

Phase 1 involved creating all the fundamental components necessary for the project, such as the registers, ALU, a bus, and the data path. There were 32 registers in total, all 32-bit, there where R0 to R15, PC, IR, MAR, HI, LO, Y, and Z. The ALU was then made by creating each operation with detail, such as using Booth's multiplication algorithm for multiplication and a restoring division algorithm for division. Most of the phase was then spent connecting all the components to the data path and ensuring everything was communicating correctly. A test bench was then made for each ALU operation to ensure everything was working as intended before advancing to the next phase.

In phase 2, the team started implementing the remaining Mini SRC datapath modules to allow for instruction execution. In this phase, the team created the Select and Encode logic, CONFF branching logic, memory subsystem, and input/output ports. The team also switched the design of the project to a 3-bus design to optimize instruction execution flow, make resource usage more efficient, and simplify the data path. To test the instruction execution, the following instructions were used: ld, ldi, st, addi, brzr, brnz, brpl, brmi, jr, jal, mfhi, mflo, in, and out.

The team then created a control unit in phase 3 that involved a state machine to help transition between states. A variety of instructions needed to be decoded for phase 3. Op codes for each instruction needed to be set at the right state or clock cycle to ensure the correct functionality. Each clock cycle would also need to update the correct control signals based on the current instructions. Setting up these control signals and testing them was done by using Intel ModelSim to check for the correct instruction flow.

The final phase, phase 4, required the team to test the proposed design on a dedicated FPGA. However, due to time constraints, the team was unable to test in on an actual FPGA. Instead, the design was simulated using Intel ModelSim by running a given set of instructions. These instructions would give the team their results and would allow for a better understanding of how the solution would perform in a real-world test.

In the future, the team will continue the project by testing on a dedicated FPGA to ensure that all functionality is working as intended. Once the design is simulated on an FPGA, future improvements such as reducing instruction fetch cycles, optimizing clock cycles, and any other enhancements to make the CPU more efficient can be implemented.

Table of Contents

Abstract.....	2
Problem Specification	5
Problem Design and Implementation	6
Evaluation Results	7
Discussion.....	7
Conclusion and future work.....	8
Appendix	9
Verilog code	9
Phase 3 Simulation Results.....	19
Phase 3 Memory Contents	37
Phase 4 Memory Contents	41

Table of Figures

Figure 1 – 32-bit register module.	9
Figure 2 – MDR register module.....	9
Figure 3 – Bus A of the three-bus system.	10
Figure 4 – Bus B of the three-bus system.	10
Figure 5 – The Datapath of the CPU.	11
Figure 6 – ALU module.	12
Figure 7 – Full Adder used for Carry Lookahead Adder.	12
Figure 8 – 4-bit Carry Lookahead Adder.	13
Figure 9 – 32-bit Carry Lookahead Adder	13
Figure 10 – and or logic used for ALU operations.	13
Figure 11 – Booth pairs multiplication algorithm.	14
Figure 12 – Restorative Division Algorithm.	14
Figure 13 – negate and not ALU operations.	15
Figure 14 – shift right, right arithmetic, left ALU operations.	15
Figure 15 – left and right rotate ALU operations.	16
Figure 16 – RAM module.	16
Figure 17 – CONFF branching logic.....	17
Figure 18 – Select Encode logic for the CPU.....	17
Figure 19 – Control Unit logic for the CPU.	18
Figure 20 – Seven Segment Display logic for Phase 4.....	19
Figure 21 – Ldi R2, 0x69 ; R2 = 0x69 Ldi R2, 2(R2) ; R2 = 0x6B	19
Figure 22 – Ld R1, 0x47 ; R1 = (0x47) = 0x94	20
Figure 23 – Ldi R1, 1(R1) ; R1 = 0x95	20

Figure 24 – ld R0, -7(R1) ; R0 = (0x8E) = 0x34.....	21
Figure 25 – ldi R3, 3 ; R3 = 3 ldi R2, 0x43 ; R2 = 0x43.....	21
Figure 26 – brmi R2, 3 ; continue with the next instruction (will not branch)	22
Figure 27 – ldi R2, 6(R2) ; R2 = 0x49	22
Figure 28 – ldi R2, 6(R2) ; R2 = 0x49	23
Figure 29 – ld R7, -2(R2) ; R7 = (0x49 - 2) = 0x94	23
Figure 30 – nop.....	24
Figure 31 – brpl R7, 2; ldi R5, 4(R2); ldi R4, -3(R5);	24
Figure 32 – add R2, R2, R3 ; R2 = 0x4C	25
Figure 33 – addi R7, R7, 3 ; R7 = 0x97	25
Figure 34 – neg R7, R7 ; R7 = 0xFFFFF69.....	26
Figure 35 – not R7, R7 ; R7 = 0x96	26
Figure 36 – andi R7, R7, 0xF ; R7 = 6.....	27
Figure 37 – ror R1, R0, R3 ; R1 = 0x80000006.....	27
Figure 38 – ori R7, R1, 9 ; R7 = 0x8000000F	28
Figure 39 – shra R1, R7, R3 ; R1 = 0xF0000001.....	28
Figure 40 – shr R2, R2, R3 ; R2 = 9	29
Figure 41 – st 0x8E, R2 ; (0x8E) = 9 rol R2, R0, R3 ; R2 = 0x1A0	29
Figure 42 – or R4, R3, R0 ; R4 = 0x37	30
Figure 43 – and R1, R2, R0 ; R1 = 0x20.....	30
Figure 44 – st 0x27(R1), R4 ; (0x47) = 0x37 sub R0, R2, R4 ; R0 = 0x169	31
Figure 45 – shl R1, R2, R3 ; R1 = 0xD00 ldi R4, 6 ; R4 = 6	31
Figure 46 – ldi R5, 0x1B ; R5 = 0x1B.....	32
Figure 47 – mul R5, R4 ; HI = 0; LO = 0xA2	32
Figure 48 – mfhi R7; R7 = 0	33
Figure 49 – mflo R6; R6 = 0xa2.....	33
Figure 50 – div R5, R4; HI = 3, LO = 4. Also Ldi R10, 1(R4); R10 = 7	34
Figure 51 – ldi R11, -2(R5); R11 = 0x19	34
Figure 52 – ldi R12, 0(R6); R12 = 0xa2	35
Figure 53 – ldi R13, 3(R7); R13 = 3.....	35
Figure 54 – jal R12	36
Figure 55 – halt and ORG 0xA2 and sub R8, R11, R13; R9 = 0xA9.....	36
Figure 56 – sub R8, R11, R13; R8 = 0x16.....	37

Problem Specification

The purpose of the CPU design project is to design, simulate, and implement a simple RISC-style computer consisting of a RISC processor, memory, and I/O. The project will be primarily developed using the Intel Quartus Prime software and simulated using the ModelSim program. The specific chip that will implement the design is Cyclone V of the DE0-CV development board.

The Mini SRC must be implemented as a 32-bit machine with a 32-bit datapath and sixteen 32-bit registers. Within these registers, there must be 8 general-purpose registers, two return value registers, four argument registers, a stack pointer, and a return address register. The Mini SRC must also implement a program counter, instruction register, memory address register, memory data register, and finally a HI and LO register for storing 64-bit values from certain ALU operations. The memory must be capable of holding 512 words.

The project can be broken down into four phases:

1. **Phase 1:** Design the registers R0 to R15, PC, IR, MAR, HI, LO, Y, and Z. Implement a bus, datapath, and MDR register for connection to the memory module. Design an ALU for the and, or, not, neg, add/sub, mul, div, shr, shra, shl, ror, and rol instructions.
2. **Phase 2:** Design the remaining Mini SRC datapath modules, including the Select and Encode logic, CONFF branching logic, memory subsystem, and input/output ports. The designs must then be tested through the following instructions: ld, ldi, st, addi, brzr, brnz, brpl, brmi, jr, jal, mfhi, mflo, in, and out.
3. **Phase 3:** Design and functionally simulate the control unit of the Mini SRC. This involves making a state machine that continuously updates the various control signals of the processor to perform the intended instruction. Phase 3 also introduces the nop and halt instructions.
4. **Phase 4:** Simulate the processor on a dedicated FPGA through display units, clock circuitry, and I/O pins.

There are a variety of constraints that must be accounted for in the CPU design project. These constraints can be viewed below:

1. **Technology:** The project must utilize the Cyclone V (5CEBA4F23C7) chip, which has many limitations, such as pin count and processing capacity. Although it was sufficient for this task, the chip still needs to be taken into consideration when making design choices.
2. **Software:** The Mini SRC must be designed and implemented using the Quartus Prime software and the VHDL or Verilog coding languages. The processor must be tested and simulated using the Intel ModelSim software.
3. **Instruction Set/Formatting:** The MINI SRC must implement a predefined set of instructions provided by the course. These instructions must function in a particular way and follow a specific format.

Problem Design and Implementation

The specific Mini SRC design that we implemented has a few key unique features. The most noteworthy aspect of our design is the three-bus architecture that is the core of the Datapath. The first bus, referred to as Bus A, takes the 16 general-purpose registers and the constant sign extended register as inputs and outputs one of these registers to the first input of the ALU. The second bus, referred to as Bus B, takes the same 16 general-purpose registers as A alongside the PC, MDR, HI, LO, and IN registers as inputs and outputs one of these to the second ALU input wire. Finally, the third bus, referred to as Bus C, takes the output of the ALU and delivers it to any of the previously mentioned registers, depending on the instruction.

Within the Datapath, both Bus B and Bus A are instantiated and attached to their respective registers, while the C Bus is just a wire within the Datapath that serves as an input for all registers and an output for the ALU. Overall, this design reduces the number of registers by eliminating the Y and Z registers and saves 2 clock cycles on every instruction by eliminating the cycles needed to load and read the Y and Z registers.

The control unit uses the second suggested method of assigning logical statements to the control signals based on state indicators of which instruction and clock cycle are currently being executed. The control unit manages all signals for the registers, buses, ALU, and memory subsystems so that instructions can be read from memory and executed correctly. In phase 4, a hex-digit encoder was implemented as an output interface, but any potential encoding of input or output information could be managed by the CPU with the implementation of the appropriate encoding or decoding logic.

Each module in the final design is well defined by the set of inputs that it takes to produce certain outputs and the logic it uses to do so. Besides the control unit, the most important modules are the memory subsystem, the ALU, and the additional control logic. The memory subsystem consists of two registers (the memory data register and the memory address register) and the memory itself. The MAR is used to store the address that is being read or written, and the MDR stores the data to be written or the data that has been read. The ALU simply takes inputs from bus A and B, along with an opcode to select a specific operation, to output a result to bus C, which is connected to the input of most of the registers in the design. Finally, the additional control logic is made up of the CONFF module and the select encode logic. The CONFF module is dedicated to the branch instruction, and it checks if the branch condition (zero, non-zero, positive, or negative) is met by a specified register and sends an enable signal to the program counter. The select encode logic decodes the register indexes in the instruction register and provides the appropriate enable and output signals for the appropriate registers. All these subsystems work together to perform operations on the memory and registers, implementing the instruction set given in the CPU specification.

Any properly written machine code loaded into memory will be correctly executed on the Mini SRC in our final design. First, an instruction is fetched from memory and placed into the instruction register. Next, the information in the instruction is used to determine which operation is being done on which registers and with which constants (if any). After instruction execution, the Mini SRC fetches the next instruction and continues execution until a halt instruction is encountered or it

receives an external stop signal. Almost every instruction in the instruction set operates by first fetching the instruction, then performing an ALU operation that is then stored in some register, and finally some instructions interact with the memory subsystem after the ALU operation.

Evaluation Results

The Mini SRC design was made to perform on the 50 MHz clock of the DE0 board, however it could theoretically operate at higher frequencies. The maximum frequency of operation would be dependent on the timing delays of the specific logic used to implement the functional description from the Verilog code. The Cycles per instruction is constant in the current design, with every instruction always taking six clock cycles, however minor changes to the instruction fetch could reduce this to five cycles for every instruction. The current design does not fully fit on the DE0 board due to the functional design needing more LABs (a basic logic building block) than the board can provide. Doing a compilation with place and route while excluding the full size of the RAM, the design uses 32% of the available logic circuitry (ALMs). Overall, the design meets the specifications outlined in the lab handouts and it performs better than required, however there are still many optimizations that could be made to improve resource usage and speed.

Discussion

Designing all the features required for the project in phase 1 was completed with few issues. The team ran into their first problem when it came to combining all the individual modules to work together. With so many different components to connect, it was difficult to keep track of what was going on. Additionally, figuring out which components were communicating successfully during testing required careful debugging and coordination. With all components connected, phase 1 became one complex system, making debugging each testbench very challenging. With the current architecture, every ALU operation requires its own test bench, making debugging and simulating tedious and hard to observe. After several rounds of debugging and pinpointing the components that were functioning incorrectly, the team quickly progressed through the remaining test benches, resolving issues as they arrived. Despite all these complexities, the team got all components functioning correctly for phase 1.

In phase 2, to improve the overall design of the project, the team implemented a 3-bus design. A 3-bus design would result in reduced clock cycles to execute instructions, optimized instruction execution flow, efficient resource usage, a simplified data path, and more flexibility for any future implementation. However, one issue caused by converting the design was modifying all the provided test bench control signals to work with this change. With bus A and B acting as inputs for the ALU and bus C acting as the output, a lot of changes to the test benches were made to accommodate this new architecture. With this change, the Z and Y registers were eliminated, and each test bench needed to be shortened by 2 clock cycles. Additionally, further debugging needed to be done on each bus as the team ran into errors caused by the individual buses not working as intended. Once each test bench was changed to handle the new architecture, the team was able to successfully implement the remainder of phase 2.

For phase 3, implementing the instruction set architecture posed a couple of challenges that the team needed to solve. The first was decoding all the instructions to figure out the correct op codes. This process took a lot of time, with each op code needing to be tested individually. The control signals would then need to be correctly set up to ensure the correct instruction was running at the right time. To solve these problems, the team designed a control unit that would handle each state of the operations individually using a state machine. The control unit would utilize the data path and ensure the correct instruction execution flow. Depending on the state and op code, certain control signals would be enabled to perform the correct instruction. Further testing was done using Intel ModelSim to evaluate the clock cycles and determine whether the instruction flow was correct. If an error was encountered, each clock cycle for that operation would need to be looked over, resulting in a lot of time being dedicated to debugging. Eventually, the team was able to get all instructions working, resulting in a successful control unit.

In Phase 4, the team was supposed to run the design on a dedicated FPGA. However, due to time constraints, this could not be completed. Instead, the design was simulated using Intel ModelSim by running the provided set of instructions. This simulation would allow the team to better understand how the proposed design would handle a real-world application. The results can then be used to make any future adjustments and help set up for real-world testing.

Conclusion and future work

In conclusion, although there were many challenges during the design and implementation of each phase, the resulting CPU worked as intended and achieved most of its required functionality for this project. Furthermore, due to additional features such as the carry-lookahead adder and 3-bus system, the CPU performance was better than the initial requirements and was well designed for clock cycle efficiency. Phase 4 was the sole objective of this project that was not fully complete, having never made it onto the DEO-CV development board; however, the simulation functioned correctly apart from the hex-display.

In the future, there are many optimizations that could be made to improve the efficiency and performance of the CPU.

First, the current instruction fetch uses three clock cycles to increment the program counter, retrieve the desired instruction from memory, and pass through the MDR to the instruction register for processing. This could be reduced to 2 clock cycles as opposed to 3 by modifying the program counter increment. Specifically, the control signals PCout, PCen, and the incrementing ALU operation can be combined into one clock cycle, reducing the overall clock cycles by one for every instruction. Overall, this would reduce the CPI and increase the performance of the CPU.

Second, all instructions currently take 6 clock cycles to execute; however, some instructions do not require all six to function. This means that shorter instructions are currently using all clock cycles but only modifying control signals for some of them. In the future, it would be beneficial to implement an algorithm that skips these unnecessary clock cycles for the shorter instructions. This would reduce the number of excess clock cycles being used, improving CPU performance and efficiency.

Third, the control unit currently utilizes the second method when generating the necessary control signals; however, in the future, it would be advisable to implement the first method for a variety of reasons. Currently, with method 2 as the control unit, it would be difficult to add new instructions to the architecture as the logic for each control signal may change. So, if the project were to be expanded upon, the current architecture would be tedious to change. Furthermore, the first method will have cleaner and more efficient code because of its simplicity and avoidance of combinational logic expressions for each control signal, with the only drawback being hardware usage.

Appendix

Verilog code

```

1  module reg32 (
2      input clr, clk, en,
3      input [31:0] d,
4      output wire [31:0] q
5  );
6      reg [31:0] f;
7      initial f = 32'b0;
8      always @ (posedge clk)
9      begin
10         if (clr) begin
11             f <= 32'b0;
12         end
13         else if (en) begin
14             f <= d;
15         end
16     end
17     assign q = f[31:0];
18 endmodule
19

```

Figure 1 – 32-bit register module.

```

1  module mdr_reg (
2      input clr, clk, MDRen, read,
3      input [31:0] Mdatain,
4      input [31:0] BusMuxout,
5      output wire [31:0] q
6  );
7
8      reg [31:0] f;
9      reg32 r(clr, clk, MDRen, f, q);
10     always @ (*) begin
11         if (read) f = Mdatain;
12         else f = BusMuxout;
13     end
14 endmodule
15

```

Figure 2 – MDR register module.

```

1 module bussyA (
2   input [31:0] BMir0, BMir1, BMir2, BMir3, BMir4, BMir5, BMir6, BMir7, BMir8, BMir9, BMir10,
3   BMir11, BMir12, BMir13, BMir14, BMir15, CSignExt,
4   |
5   input [15:0] rout,
6   input Cout,
7   |
8   output wire [31:0] BusMuxout
9 );
10
11   reg [31:0]q;
12
13   always @ (*) begin
14     if (rout[0]) q = BMir0;
15     if (rout[1]) q = BMir1;
16     if (rout[2]) q = BMir2;
17     if (rout[3]) q = BMir3;
18     if (rout[4]) q = BMir4;
19     if (rout[5]) q = BMir5;
20     if (rout[6]) q = BMir6;
21     if (rout[7]) q = BMir7;
22     if (rout[8]) q = BMir8;
23     if (rout[9]) q = BMir9;
24     if (rout[10]) q = BMir10;
25     if (rout[11]) q = BMir11;
26     if (rout[12]) q = BMir12;
27     if (rout[13]) q = BMir13;
28     if (rout[14]) q = BMir14;
29     if (rout[15]) q = BMir15;
30     if (Cout) q = CSignExt;
31   end
32   assign BusMuxout = q[31:0];
33
34 endmodule
35
36

```

Figure 3 – Bus A of the three-bus system.

```

1 module bussyB (
2   input [31:0] BMir0, BMir1, BMir2, BMir3, BMir4, BMir5, BMir6, BMir7, BMir8, BMir9, BMir10,
3   BMir11, BMir12, BMir13, BMir14, BMir15, BMIHI, BMILO, BMIIPC, BMIMDR, BMIInport,
4   input [15:0] rout,
5   input HIout, LOout, PCout, MDROUT, Inportout, |
6   output wire [31:0] BusMuxout
7 );
8
9   reg [31:0]q;
10
11   always @ (*) begin
12     if (rout[0]) q = BMir0;
13     if (rout[1]) q = BMir1;
14     if (rout[2]) q = BMir2;
15     if (rout[3]) q = BMir3;
16     if (rout[4]) q = BMir4;
17     if (rout[5]) q = BMir5;
18     if (rout[6]) q = BMir6;
19     if (rout[7]) q = BMir7;
20     if (rout[8]) q = BMir8;
21     if (rout[9]) q = BMir9;
22     if (rout[10]) q = BMir10;
23     if (rout[11]) q = BMir11;
24     if (rout[12]) q = BMir12;
25     if (rout[13]) q = BMir13;
26     if (rout[14]) q = BMir14;
27     if (rout[15]) q = BMir15;
28     if (PCout) q = BMIIPC;
29     if (HIout) q = BMIHI;
30     if (LOout) q = BMILO;
31     if (MDROUT) q = BMIMDR;
32     if (Inportout) q = BMIInport;
33   end
34   assign BusMuxout = q[31:0];
35
36 endmodule
37

```

Figure 4 – Bus B of the three-bus system.

```

1 module datapath(
2     input clk, clr, stop,
3     output wire [7:0] hexOut1, hexOut2,
4     output wire [31:0] BMir0, BMir1, BMir2, BMir3, BMir4, BMir5, BMir6, BMir7, BMir8, BMir9,
5     BMir10, BMir11, BMir12, BMir13, BMir14, BMir15, BMIHI, BMILO, BMIPC, BMIMDR, IRout, memChip
6 );
7 wire HIout, LOout, PCout, MDRout, Inportout, Cout;
8 wire HIen, LOen, PCen, MAREn, IREN, Outporten, RAMwrite;
9 wire [4:0] op;
10 wire MDRen, Read, Graen, Graout, Grb, Grc, BAout, CON, reten, run;
11 wire [31:0] Input;
12
13 assign Input = 32'h80;
14
15 wire [31:0] BMIInport, CSignExt, mData, Output, bussyAout, bussyBout;
16 wire [63:0] bussyC;
17 wire [15:0] Raout, Rbout, Rcout, Raen;
18
19 wire CFFout;
20
21 controlUnit control(Graen, Graout, Grb, Grc,
22 HIout, LOout, PCout, MDRout, Inportout, Cout,
23 HIen, LOen, PCen, MAREn, IREN, Outporten, RAMwrite, MDRen, Read, BAout, CON, reten, run,
24 op, clk, clr, stop, IRout[31:27]);
25
26 reg32 r0(clr, clk, Raen[0], bussyC[31:0], BMir0);
27 reg32 r1(clr, clk, Raen[1], bussyC[31:0], BMir1);
28 reg32 r2(clr, clk, Raen[2], bussyC[31:0], BMir2);
29 reg32 r3(clr, clk, Raen[3], bussyC[31:0], BMir3);
30 reg32 r4(clr, clk, Raen[4], bussyC[31:0], BMir4);
31 reg32 r5(clr, clk, Raen[5], bussyC[31:0], BMir5);
32 reg32 r6(clr, clk, Raen[6], bussyC[31:0], BMir6);
33 reg32 r7(clr, clk, Raen[7], bussyC[31:0], BMir7);
34 reg32 r8(clr, clk, Raen[8], bussyC[31:0], BMir8);
35 reg32 r9(clr, clk, Raen[9], bussyC[31:0], BMir9);
36 reg32 r10(clr, clk, Raen[10], bussyC[31:0], BMir10);
37 reg32 r11(clr, clk, Raen[11], bussyC[31:0], BMir11);
38 reg32 r12(clr, clk, Raen[12], bussyC[31:0], BMir12);
39 reg32 r13(clr, clk, Raen[13], bussyC[31:0], BMir13);
40 reg32 r14(clr, clk, Raen[14], bussyC[31:0], BMir14);
41 reg32 r15(clr, clk, Raen[15] | reten, bussyC[31:0], BMir15);
42 reg32 PC(clr, clk, (PCen | (CON && CFFout)), bussyC[31:0], BMIPC);
43 reg32 MAR(clr, clk, MAREn, bussyC[31:0], memChip);
44 reg32 IR(clr, clk, IREN, bussyC[31:0], IRout);
45 reg32 HI(clr, clk, HIen, bussyC[63:32], BMIHI);
46 reg32 LO(clr, clk, LOen, bussyC[31:0], BMILO);
47 reg32 Outport(clr, clk, Outporten, bussyAout, Output);
48 reg32 Inport(clr, clk, 1, Input, BMIInport);
49 mdr reg mdr(clr, clk, MDRen, Read, mData, bussyC[31:0], BMIMDR);
50
51 //Bus
52 bussyB b(BAout ? 32'd0 : BMir0, BMir1, BMir2, BMir3, BMir4, BMir5, BMir6, BMir7, BMir8, BMir9,
53 BMir10, BMir11, BMir12, BMir13, BMir14, BMir15, BMIHI, BMILO, BMIPC, BMIMDR, BMIInport,
54 Rbout, HIout, LOout, PCout, MDRout, Inportout,
55 bussyBout);
56
57 ALU a(bussyAout, bussyBout, op, bussyC);
58
59 RAM ram(BMIMDR, memChip[8:0], Read, RAMwrite, clk, mData);
60
61 SelectEncode SelectEncodeInst(IRout, Graen, Graout, Grb, Grc, Raen, Raout, Rbout, Rcout, CSignExt);
62
63 CONFF CFF(clk, bussyC[31:0], IRout[22:19], CFFout);
64
65 bussyA BA(BMir0, BMir1, BMir2, BMir3, BMir4, BMir5, BMir6, BMir7, BMir8, BMir9,
66 BMir10, BMir11, BMir12, BMir13, BMir14, BMir15, CSignExt,
67 Raout | Rcout, Cout, bussyAout);
68
69 Seven_Seg_Display_Out s1(hexOut1, clk, Output[3:0]);
70 Seven_Seg_Display_Out s2(hexOut2, clk, Output[7:4]);
71
72 endmodule

```

Figure 5 – The Datapath of the CPU.

```

1  module ALU(input wire [31:0] A, B, input wire[4:0] op, output wire[63:0] res);
2      wire [31:0] and_result, or_result, neg_result, not_result, add_result, sub_result,
3      shr_result, shra_result, shl_result, shla_result,
4      ror_result, rol_result, incPC_result;
5
6      wire[63:0] mul_result, div_result;
7      reg [63:0] result;
8
9      and_or and_inst(A, B, 1, and_result);
10     and_or or_inst(A, B, 0, or_result);
11     notOp not_inst(B, not_result);
12     neg neg_inst(B, neg_result);
13     CLA32 add_inst(A, B, 0, add_result);
14     CLA32 sub_inst(B, (~A), 1, sub_result);
15     mul32 mul_inst(A, B, mul_result);
16     shl shl_inst(B, A, shl_result);
17     shr shr_inst(B, A, shr_result);
18     shra shra_inst(B, A, shra_result);
19     div32 div_instance(A, B, div_result);
20     ror ror_instance(B, A, ror_result);
21     rol rol_instance(B, A, rol_result);
22     CLA32 inc_instance(32'd1, B, 0, incPC_result);
23
24     always @(*) begin
25         case(op)
26             0 : result = and_result;
27             1 : result = or_result;
28             2 : result = neg_result;
29             3 : result = not_result;
30             4 : result = $signed(add_result);
31             5 : result = $signed(sub_result);
32             6 : result = mul_result;
33             7 : result = div_result;
34             8 : result = shr_result;
35             9 : result = shra_result;
36             10 : result = shl_result;
37             11 : result = ror_result;
38             12 : result = rol_result;
39             13 : result = incPC_result;
40             14 : result = B;
41             15 : result = A;
42
43             default: result = or_result;
44         endcase
45     end
46     assign res = result;
47 endmodule

```

Figure 6 – ALU module.

```

1  module fullAdder(input wire [0:0] C, X, Y, output wire [0:0] carry, sum);
2
3      assign carry = (Y & C) | (X & C) | (X & Y);
4      assign sum = X ^ Y ^ C;
5
6  endmodule

```

Figure 7 – Full Adder used for Carry Lookahead Adder.

```

1 module CLA4(input wire [3:0] A, B, input wire Cin, output wire Cout, output wire [3:0] Sum);
2
3     wire [4:0] C;
4     wire [3:0] G, P, S;
5     wire carry;
6
7     fullAdder fullAdder0(C[0], A[0], B[0], carry, S[0]);
8     fullAdder fullAdder1(C[1], A[1], B[1], carry, S[1]);
9     fullAdder fullAdder2(C[2], A[2], B[2], carry, S[2]);
10    fullAdder fullAdder3(C[3], A[3], B[3], carry, S[3]);
11
12    assign G[0] = A[0] & B[0];
13    assign G[1] = A[1] & B[1];
14    assign G[2] = A[2] & B[2];
15    assign G[3] = A[3] & B[3];
16
17    assign P[0] = A[0] | B[0];
18    assign P[1] = A[1] | B[1];
19    assign P[2] = A[2] | B[2];
20    assign P[3] = A[3] | B[3];
21
22    assign C[0] = Cin;
23    assign C[1] = G[0] | (P[0] & C[0]);
24    assign C[2] = G[1] | (P[1] & C[1]);
25    assign C[3] = G[2] | (P[2] & C[2]);
26    assign C[4] = G[3] | (P[3] & C[3]);
27
28    assign Cout = C[4];
29    assign Sum = S;
30
31 endmodule
32

```

Figure 8 – 4-bit Carry Lookahead Adder.

```

1 module CLA32(input wire [31:0] A, B, input wire Cin, output wire [32:0] Sum);
2
3     //wire [3:0] G0, G1, G2, G3, P0, P1, P2, P3;
4     wire [7:0] carry;
5     wire [31:0] S;
6
7     CLA4 CLA0(A[3:0], B[3:0], Cin, carry[0], S[3:0]);
8     CLA4 CLA1(A[7:4], B[7:4], carry[0], carry[1], S[7:4]);
9     CLA4 CLA2(A[11:8], B[11:8], carry[1], carry[2], S[11:8]);
10    CLA4 CLA3(A[15:12], B[15:12], carry[2], carry[3], S[15:12]);
11    CLA4 CLA4(A[19:16], B[19:16], carry[3], carry[4], S[19:16]);
12    CLA4 CLA5(A[23:20], B[23:20], carry[4], carry[5], S[23:20]);
13    CLA4 CLA6(A[27:24], B[27:24], carry[5], carry[6], S[27:24]);
14    CLA4 CLA7(A[31:28], B[31:28], carry[6], carry[7], S[31:28]);
15
16    assign C0 = carry[7];
17    assign Sum = {C0, S};
18
19
20
21 endmodule

```

Figure 9 – 32-bit Carry Lookahead Adder

```

1 module and_or(input wire [31:0] A, B, input wire selection, output wire [31:0] result);
2
3     assign result = (selection == 1)? A & B : A | B;
4
5 endmodule

```

Figure 10 – and or logic used for ALU operations.

```

1  module mul32(input wire [31:0] A, B, output wire [63:0] result);
2
3      reg signed [15:0] booth[0:15];
4      reg [63:0] sum = 64'b0;
5      integer i;
6      wire [32 + 1:0] q = {B[31], B, 1'b0};
7
8      wire [63:0] notA = $signed(~A + 1'b1);
9      wire [63:0] posA = $signed(A);
10
11  always @* begin
12      sum = 64'b0;
13      for(i = 0; i < 32; i = i+2) begin
14          if({q[i+2], q[i+1], q[i]} == 3'b000 || {q[i+2], q[i+1], q[i]} == 3'b111) booth[i/2] = 0;
15          else if ({q[i+2], q[i+1], q[i]} == 3'b001 || {q[i+2], q[i+1], q[i]} == 3'b010) booth[i/2] = 1;
16          else if ({q[i+2], q[i+1], q[i]} == 3'b101 || {q[i+2], q[i+1], q[i]} == 3'b110) booth[i/2] = -1;
17          else if ({q[i+2], q[i+1], q[i]} == 3'b011) booth[i/2] = 2;
18          else if ({q[i+2], q[i+1], q[i]} == 3'b100) booth[i/2] = -2;
19      end
20
21      for(i = 0; i <= 30; i = i+2) begin
22          if(booth[i/2] == -1) sum = sum + (notA << i);
23          else if (booth[i/2] == 1) sum = sum + (posA << i);
24          else if (booth[i/2] == 2) sum = sum + ((posA<<1)<<i);
25          else if (booth[i/2] == -2) sum = sum + ((notA<<1)<<i);
26      end
27  end
28
29
30
31  assign result = sum;
32
33  endmodule

```

Figure 11 – Booth pairs multiplication algorithm.

```

1  module div32(input wire [31:0] Q, M, output wire [63:0] result);
2
3      reg [31:0] q, A;
4      reg [31:0] posM, negM;
5      reg [63:0] r;
6      integer i, negRes;
7
8  always @* begin
9
10     if(M[31] == 1'b0) begin
11         posM = M;
12         negM = -M;
13     end
14     else begin
15         posM = -M;
16         negM = M;
17     end
18
19     if(Q[31] ^ M[31]) negRes = 1;
20     else negRes = 0;
21
22     if(Q[31] == 1'b0) q = Q;
23     else q = -Q;
24
25     A = 32'h0;
26
27     for(i = 0; i < 32; i = i+1) begin
28         A = A << 1;
29         A[0] = q[31];
30         q = q << 1;
31
32         if(A[31] == 0) A = A + negM;
33         else A = A + posM;
34
35         if(A[31] == 0) q[0] = 1'b1;
36         else q[0] = 1'b0;
37     end
38
39     if(A[31] == 1) A = A + posM;
40
41
42     if(negRes == 1) q = -q;
43
44     r[63:32] = A;
45     r[31:0] = q;
46
47 end
48
49 assign result = r;
50
51 endmodule

```

Figure 12 – Restorative Division Algorithm.

```

1  module notOp(input wire [31:0] A, output wire [31:0] result);
2
3      assign result = ~A;
4
5  endmodule
6
7  module neg(input wire [31:0] A, output wire [31:0] result);
8
9      assign result = -A;
10
11 endmodule

```

Figure 13 – negate and not ALU operations.

```

1  module shl(input wire[31:0] A, numBits, output wire[31:0] result);
2
3      assign result = A << numBits;
4
5  endmodule
6
7  module shr(input wire[31:0] A, numBits, output wire[31:0] result);
8
9      assign result = A >> numBits;
10
11 endmodule
12
13 module shra(input signed [31:0] A, numBits, output wire[31:0] result);
14
15     assign result = A >>> numBits;
16
17 endmodule

```

Figure 14 – shift right, right arithmetic, left ALU operations.

```

1  module ror(input wire[31:0] A, numBits, output wire[31:0] result);
2
3      reg bit;
4      reg[31:0] a;
5      integer i;
6
7      always @* begin
8          a = A;
9
10         for(i = 0; i < 32; i = i+1) begin
11
12             if (i < numBits) begin
13                 bit = a[0];
14                 a = a >> 1;
15                 a[31] = bit;
16             end
17         end
18     end
19 end
20
21 assign result = a;
22 endmodule
23
24 module rol(input wire[31:0] A, numBits, output wire[31:0] result);
25
26     reg bit;
27     reg[31:0] a;
28     integer i;
29
30     always @* begin
31         a = A;
32
33         for(i = 0; i < 32; i = i+1) begin
34
35             if (i < numBits) begin
36                 bit = a[31];
37                 a = a << 1;
38                 a[0] = bit;
39             end
40         end
41     end
42
43 assign result = a;
44 endmodule
45
46
47
48
49
50

```

Figure 15 – left and right rotate ALU operations.

```

1  module RAM ( input[31:0] Data, input[8:0] Address, input read, write, clk,
2      output reg [31:0] Mdatain
3  );
4
5      reg [31:0] mem[511:0];
6      initial begin
7          $readmemh("mem_init.hex",mem);
8      end
9      always @(*) begin
10         if(read) Mdatain = mem[Address];
11         else if(write) mem[Address] = Data;
12     end
13 endmodule
14
15

```

Figure 16 – RAM module.


```

1  `timescale 1ns/10ps
2  module CONFF(input clk, input[31:0] BusData, input [3:0] IR, output out);
3
4  wire [31:0] val;
5  reg tclk;
6  initial tclk <= 1;
7  always @ (negedge clk) begin
8      #1 tclk <= ~tclk;
9  end
10 reg32 r(clr, tclk, 1, BusData, val);
11
12 wire [1:0] c2 = IR[1:0];
13
14 assign out = ((c2 == 2'b00) && (val == 32'd0)) || ((c2 == 2'b01) && (val != 32'd0))
15 || ((c2 == 2'b10) && (val[31] == 0)) || ((c2 == 2'b11) && (val[31] == 1));
16
17 endmodule

```

Figure 17 – CONFF branching logic.

```

1  module SelectEncode(input wire [31:0] IR, input Graen, Graout, Grb, Grc,
2  output reg [15:0] Raen,
3  output reg [15:0] Raout, Rbout, Rcout,
4  output [31:0] C_sign_extended
5  );
6
7  always@(*) begin
8      Raen[15:0] = 'b0;
9      Raout[15:0] = 'b0;
10     Rbout[15:0] = 'b0;
11     Rcout[15:0] = 'b0;
12     if(Graout)
13         Raout[IR[26:23]]=1;
14     else if (Graen)
15         Raen[IR[26:23]]=1;
16
17
18     if(Grb)
19         Rbout[IR[22:19]]=1;
20
21     if(Grc)
22         Rcout[IR[18:15]]=1;
23 end
24
25 assign C_sign_extended = $signed(IR[18:0]);
26
27 endmodule
28

```

Figure 18 – Select Encode logic for the CPU.

```

1 module controlUnit(output reg Graen, Graout, Grb, Grc,
2 HIout, LOout, PCout, MDROUT, Inportout, Cout,
3 HIen, LOen, PCen, MAREN, IREN, Outporten, RAMwrite, MDREN, Read, BAout, CON, reten, run,
4 output reg [4:0] op,
5 input clk, clr, stop,
6 input [31:27]IR;
7
8 parameter T0 = 4'b0000, T1 = 4'b0001, T2 = 4'b0010, T3 = 4'b0011, T4 = 4'b0100, T5 = 4'b0101, T6 = 4'b0110, halt = 4'b0111;
9
10 reg S0, S1, S2, S3, S4, S5, Shalt;
11 reg [1:0] ldi_s, [1:0] ldi_s, [1:0] st_s, add_s, sub_s, shr_s, shra_s, shl_s, ror_s, rol_s, and_s, or_s, mul_s, div_s, neg_s, not_s, br_s, jr_s, jal_s, in_s, out_s, mfhi_s, mflo_s, nop_s, halt_s;
12 reg [3:0] Present_state = T0;
13 always @(posedge clk)
14 begin
15     if (clr) Present_state = T0;
16     else if (stop || halt_s) begin
17         Shalt <= 1;
18         Present_state = halt;
19     end
20     else begin
21         S0 <= 0; S1 <= 0; S2 <= 0; S3 <= 0; S4 <= 0; S5 <= 0; Shalt <= 0;
22         case (Present_state)
23             T0 : begin
24                 S0 <= 1;
25                 Present_state = T1;
26             end
27             T1 : begin
28                 S1 <= 1;
29                 Present_state = T2;
30             end
31             T2 : begin
32                 S2 <= 1;
33                 Present_state = T3;
34             end
35             T3 : begin
36                 S3 <= 1;
37                 Present_state = T4;
38             end
39             T4 : begin
40                 S4 <= 1;
41                 Present_state = T5;
42             end
43             T5 : begin
44                 S5 <= 1;
45                 Present_state = T0;
46             end
47         endcase
48     end
49 end
50
51 always @(IR) begin
52     ldi_s <= 0; ldi_s <= 0; st_s <= 0; add_s <= 0; sub_s <= 0; shr_s <= 0; shra_s <= 0; shl_s <= 0; ror_s <= 0; rol_s <= 0; and_s <= 0; or_s <= 0;
53     [add_s <= 0; and_s <= 0; or_s <= 0; mul_s <= 0; div_s <= 0; neg_s <= 0; not_s <= 0; br_s <= 0; jr_s <= 0; jal_s <= 0; in_s <= 0; out_s <= 0; mfhi_s <= 0; mflo_s <= 0; nop_s <= 0; halt_s <= 0;
54     case (IR[3:27])
55         5'b00000: ldi_s <= 1;
56         5'b00001: ldi_s <= 1;
57         5'b00010: st_s <= 1;
58         5'b00011: add_s <= 1;
59         5'b00100: sub_s <= 1;
60         5'b00101: shr_s <= 1;
61         5'b00110: shra_s <= 1;
62         5'b00111: shl_s <= 1;
63         5'b01000: ror_s <= 1;
64         5'b01001: rol_s <= 1;
65         5'b01010: and_s <= 1;
66         5'b01011: or_s <= 1;
67         5'b01100: addi_s <= 1;
68         5'b01101: andi_s <= 1;
69         5'b01110: ori_s <= 1;
70         5'b01111: mul_s <= 1;
71         5'b10000: div_s <= 1;
72         5'b10001: neg_s <= 1;
73         5'b10010: not_s <= 1;
74         5'b10011: br_s <= 1;
75         5'b10100: jr_s <= 1;
76         5'b10101: jal_s <= 1;
77         5'b10110: in_s <= 1;
78         5'b10111: out_s <= 1;
79         5'b11000: mfhi_s <= 1;
80         5'b11001: mflo_s <= 1;
81         5'b11010: nop_s <= 1;
82         5'b11011: halt_s <= 1;
83     endcase
84 end
85
86
87 always @(Clk, S0, S1, S2, S3, S4, S5, halt) begin
88     Graen <= ((ldi_s | add_s | sub_s | shr_s | shra_s | shl_s | ror_s | rol_s | and_s | or_s | mul_s | div_s | neg_s | not_s | in_s | mfhi_s | mflo_s) && S5) | (ldi_s
89     Graout <= ((st_s | jal_s | jr_s) && S4) | ((mul_s | div_s | br_s) && S3);
90     Grb <= ((ldi_s | ldi_s | jal_s | jal_s) && S4) | ((mul_s | div_s | br_s) && S3);
91     Grc <= ((add_s | sub_s | shr_s | shra_s | shl_s | ror_s | rol_s | and_s | or_s) && S3);
92     HIout <= mfhi_s && S3;
93     LOout <= mflo_s && S3;
94     PCout <= S0 | S1 | ((jal_s && S3) | (br_s && S4));
95     MDROUT <= S2 | (ldi_s && S5);
96     Inportout <= in_s && S3;
97     Cout <= ((ldi_s | ldi_s | st_s | addi_s | andi_s | ori_s) && S3) | (br_s && S4);
98     HIen <= (mul_s | div_s) && S3;
99     LOen <= (mul_s | div_s) && S3;
100     PCen <= S1 | ((jr_s | jal_s) && S4);
101     MAREN <= S0 | ((ldi_s | st_s) && S3);
102     IREN <= S2;
103     Outporten <= out_s && S3;
104     RAMwrite <= st_s && S5;
105     MDREN <= S1 | ((ldi_s | st_s) && S4);
106     Read <= S1 | (ldi_s && S4);
107     BAout <= (ldi_s | ldi_s | st_s) && S3;
108     CON <= br_s && S4;
109     reten <= jal_s && S3;
110     run <= !Shalt;
111
112     if (S1) op <= 5'd13;
113     else if ((mflo_s | mfhi_s | jal_s | in_s) && S3) | S2 | S0 | (ldi_s && S5)) op <= 5'd14;
114     else if ((st_s | jr_s | jal_s) && S4) | (br_s && S3)) op <= 5'd15;
115     else if ((andi_s | and_s) && S3) op <= 5'd0;
116     else if ((or_s | ori_s) && S3) op <= 5'd1;
117     else if (neg_s && S3) op <= 5'd2;
118     else if (not_s && S3) op <= 5'd3;
119     else if ((add_s | addi_s | ldi_s | ldi_s | st_s) && S3) | (br_s && S4)) op <= 5'd4;
120     else if (sub_s && S3) op <= 5'd5;
121     else if (mul_s && S3) op <= 5'd6;
122     else if (div_s && S3) op <= 5'd7;
123     else if (shr_s && S3) op <= 5'd8;
124     else if (shra_s && S3) op <= 5'd9;
125     else if (shl_s && S3) op <= 5'd10;
126     else if (ror_s && S3) op <= 5'd11;
127     else if (rol_s && S3) op <= 5'd12;
128
129
130 end
131
132
133
134
135 endmodule
136

```

Figure 19 – Control Unit logic for the CPU.

```

1 module Seven_Seg_Display_Out (
2   output reg [7:0] outputt, input clk,
3   input [3:0] data);
4   always @(negedge clk) begin
5     case (data[3:0])
6       4'b0000 : outputt <= 8'b11000000;
7       4'b0001 : outputt <= 8'b11111001;
8       4'b0010 : outputt <= 8'b10100100;
9       4'b0011 : outputt <= 8'b10110000;
10      4'b0100 : outputt <= 8'b10011001;
11      4'b0101 : outputt <= 8'b10010010;
12      4'b0110 : outputt <= 8'b10000010;
13      4'b0111 : outputt <= 8'b11111000;
14      4'b1000 : outputt <= 8'b10000000;
15      4'b1001 : outputt <= 8'b10010000;
16      4'b1010 : outputt <= 8'b10001000;
17      4'b1011 : outputt <= 8'b10000011;
18      4'b1100 : outputt <= 8'b11000110;
19      4'b1101 : outputt <= 8'b10100001;
20      4'b1110 : outputt <= 8'b10000110;
21      4'b1111 : outputt <= 8'b10001110;
22     endcase
23   end
24 endmodule

```

Figure 20 – Seven Segment Display logic for Phase 4.

Phase 3 Simulation Results

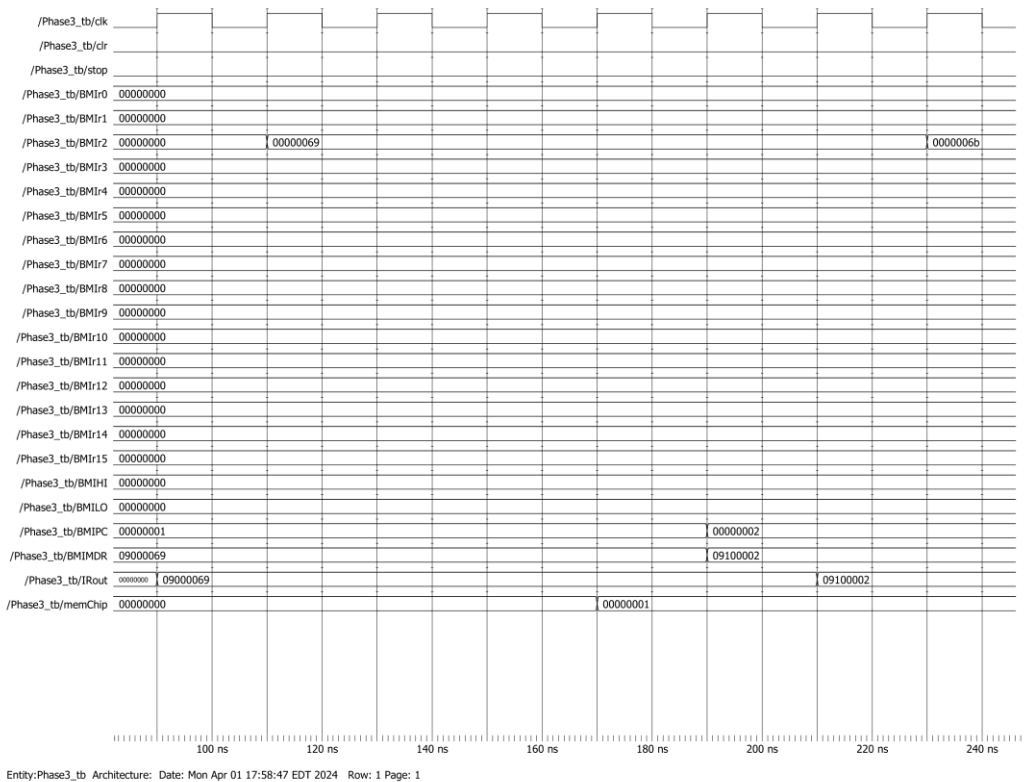


Figure 21 – `ldi R2, 0x69 ; R2 = 0x69` `ldi R2, 2(R2) ; R2 = 0x6B`

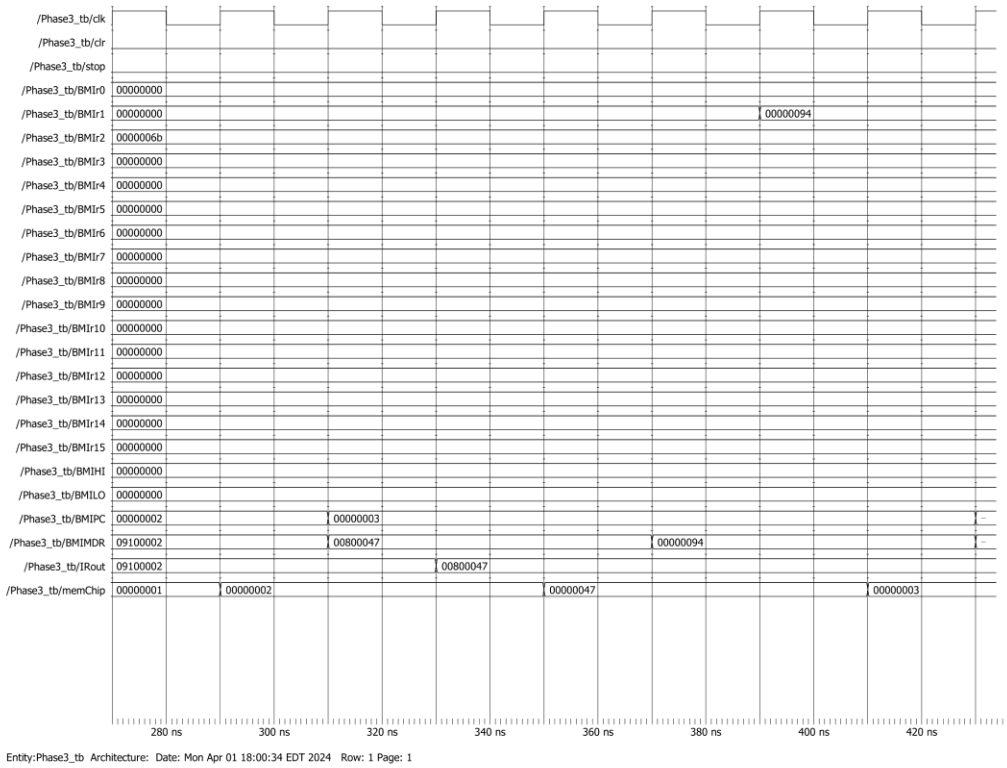


Figure 22 – ld R1, 0x47 ; R1 = (0x47) = 0x94

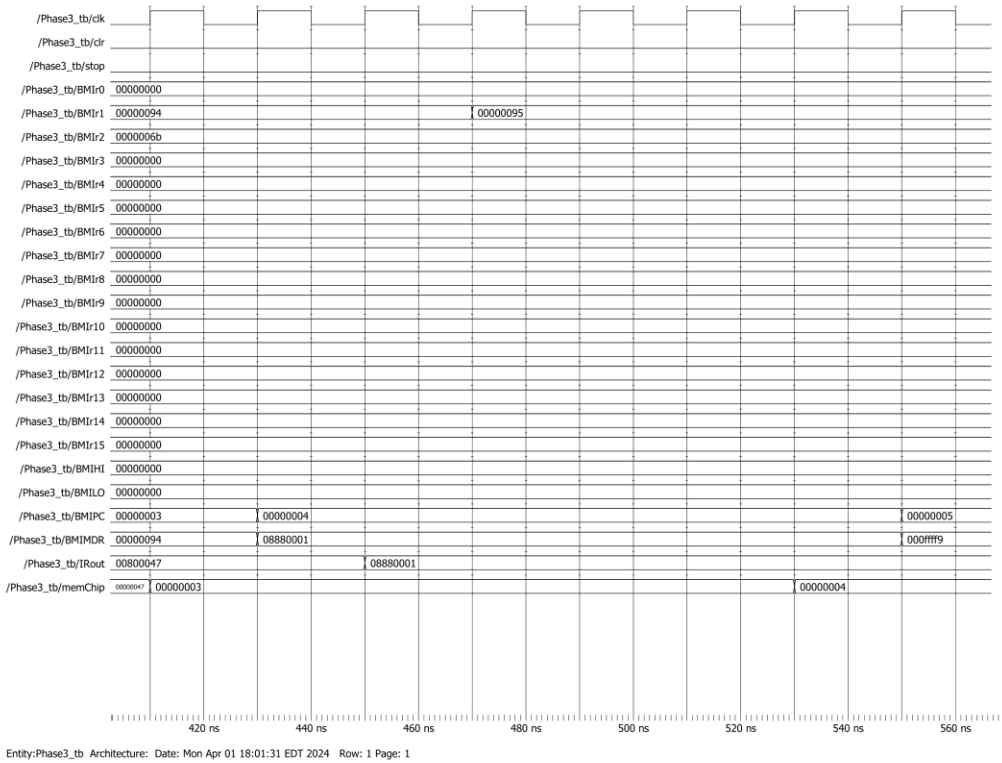


Figure 23 – ldi R1, 1(R1) ; R1 = 0x95

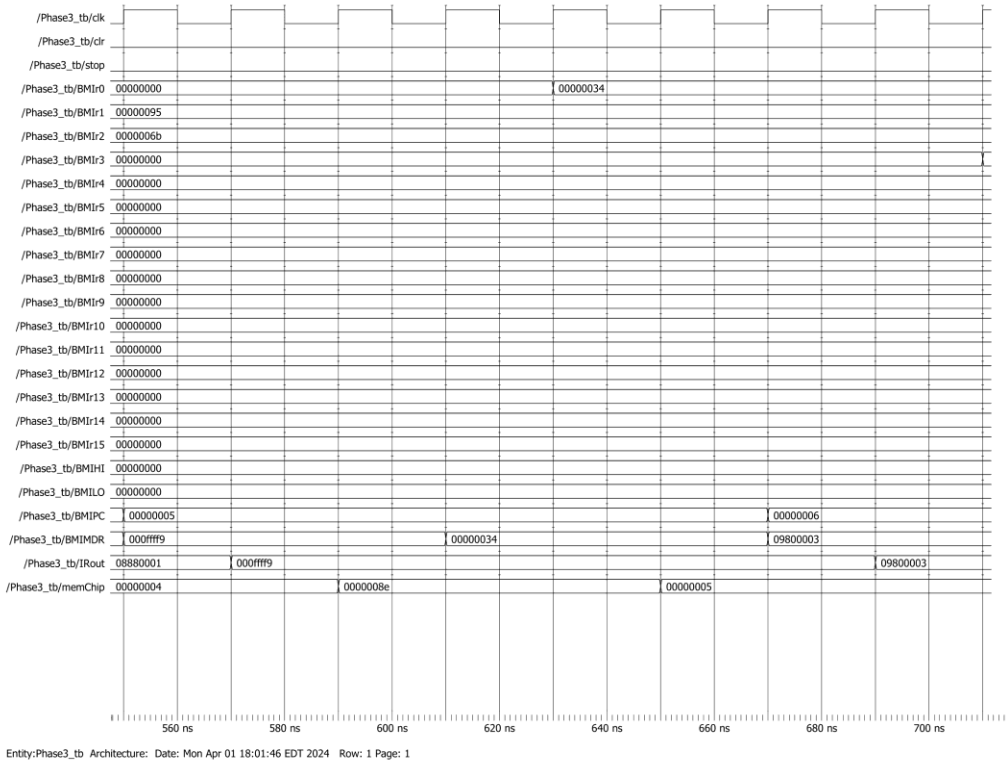


Figure 24 – $ld R0, -7(R1)$; $R0 = (0x8E) = 0x34$

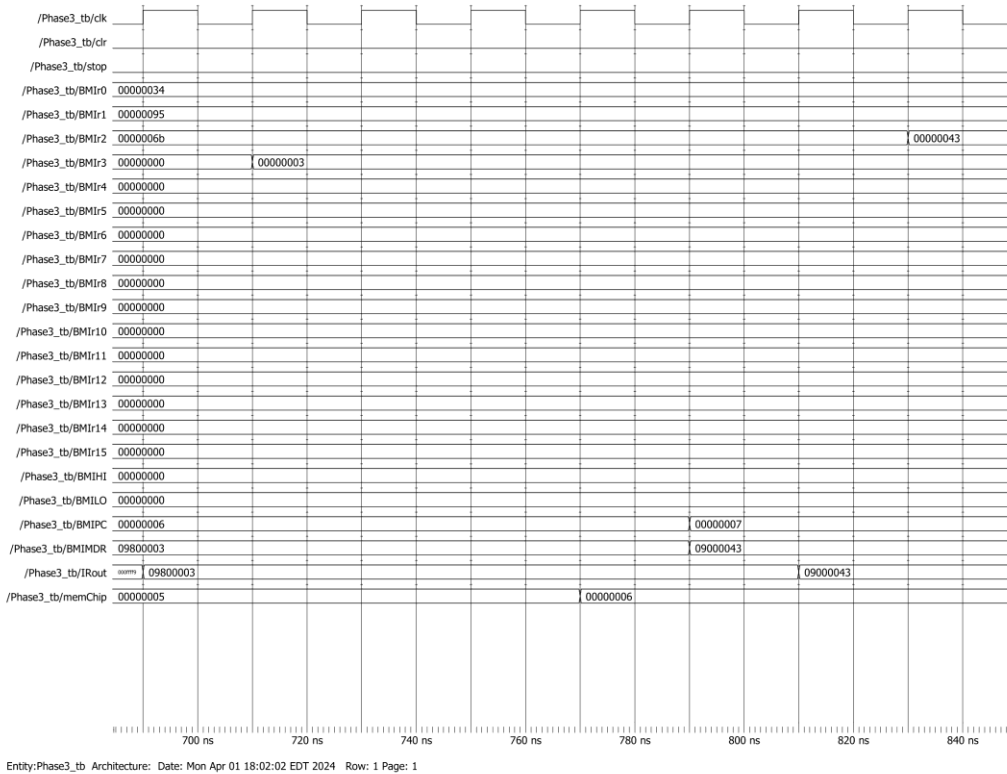


Figure 25 – $ldi R3, 3$; $R3 = 3$ $ldi R2, 0x43$; $R2 = 0x43$

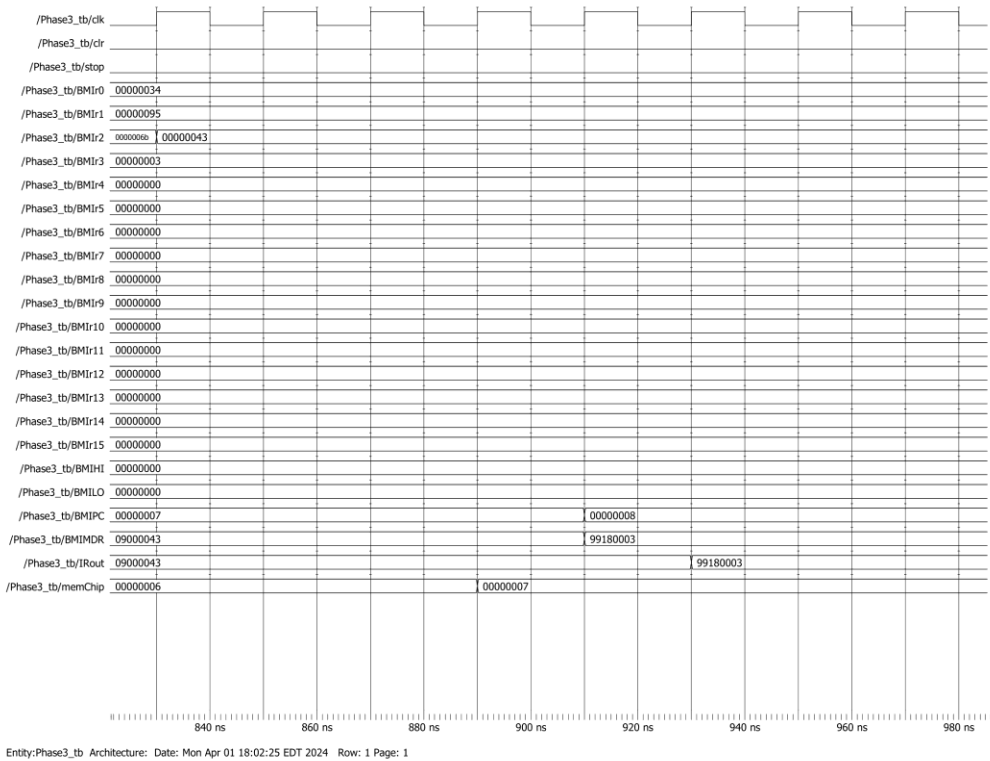


Figure 26 – brmi R2, 3 ; continue with the next instruction (will not branch)

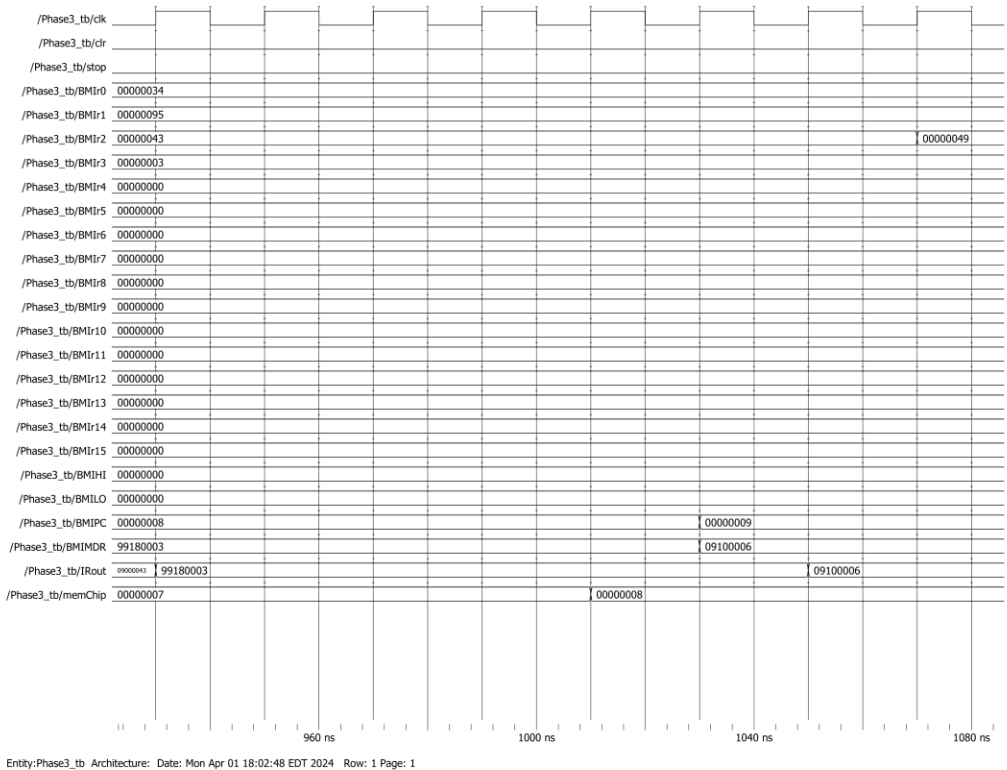


Figure 27 – ldi R2, 6(R2) ; R2 = 0x49

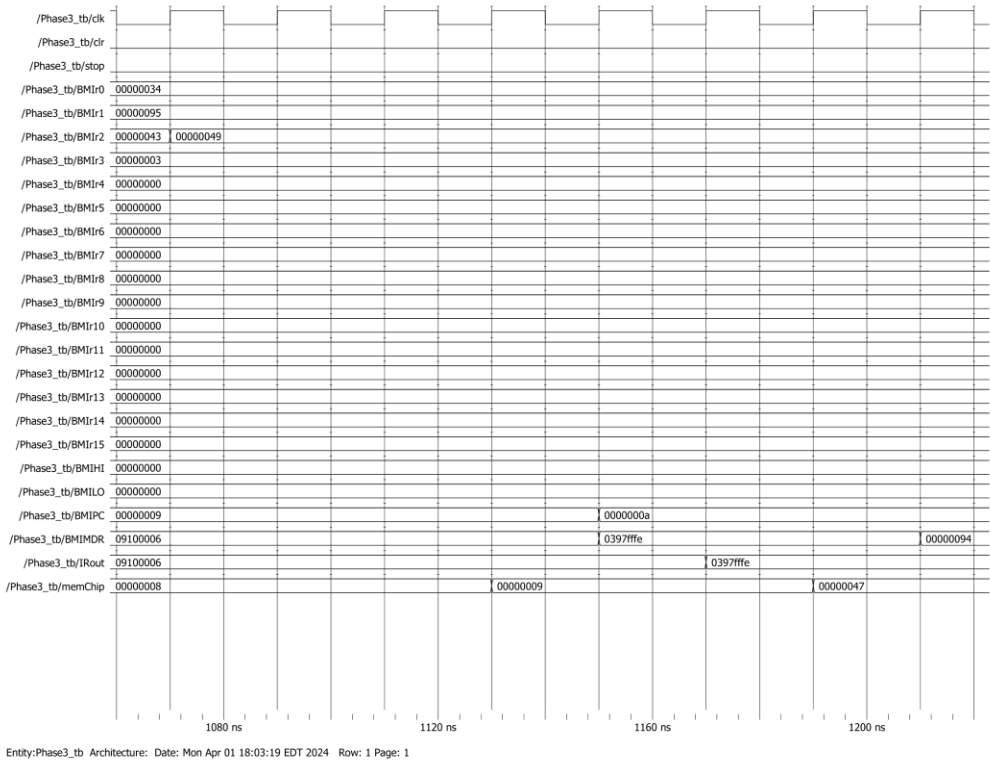


Figure 28 – *ldi R2, 6(R2) ; R2 = 0x49*

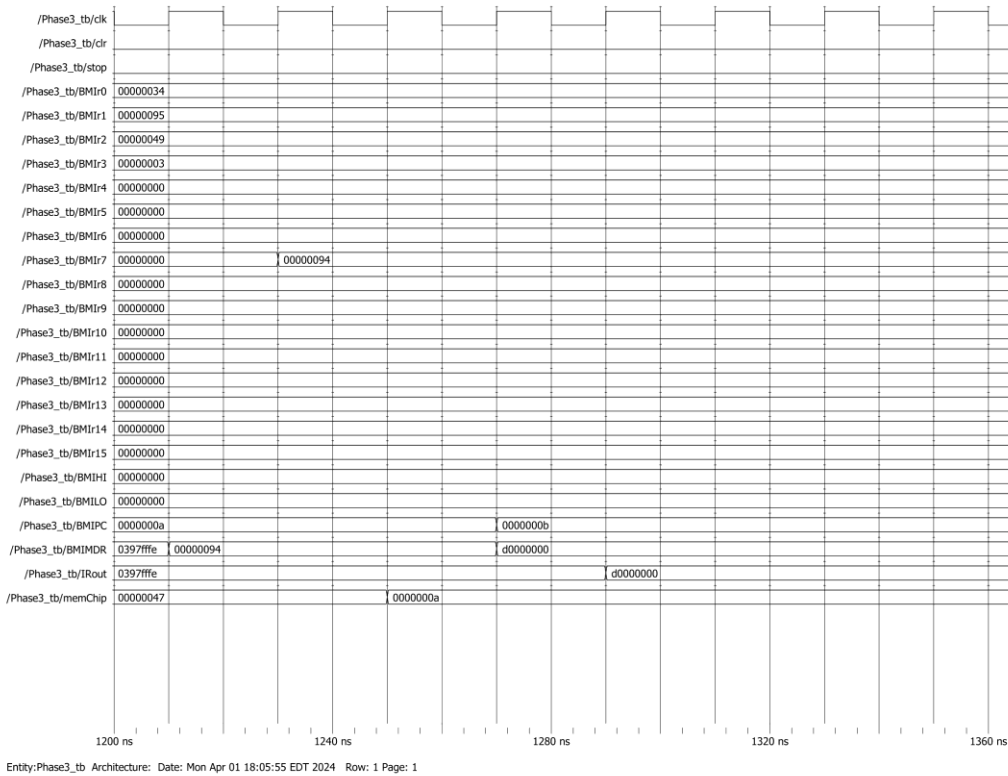


Figure 29 – *ld R7, -2(R2) ; R7 = (0x49 - 2) = 0x94*

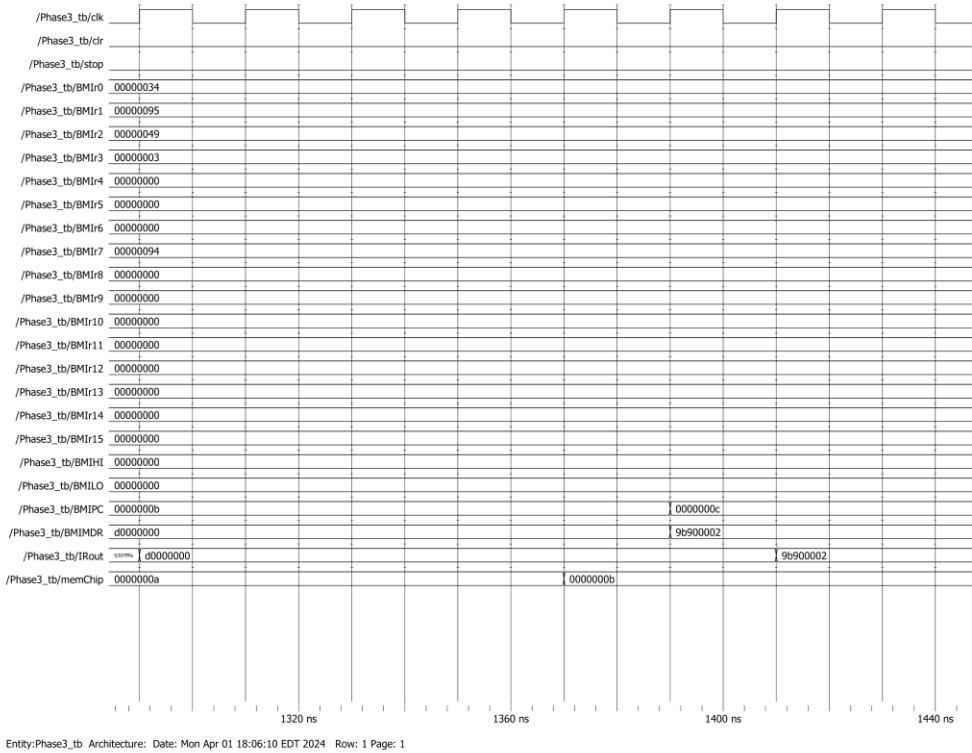


Figure 30 – nop

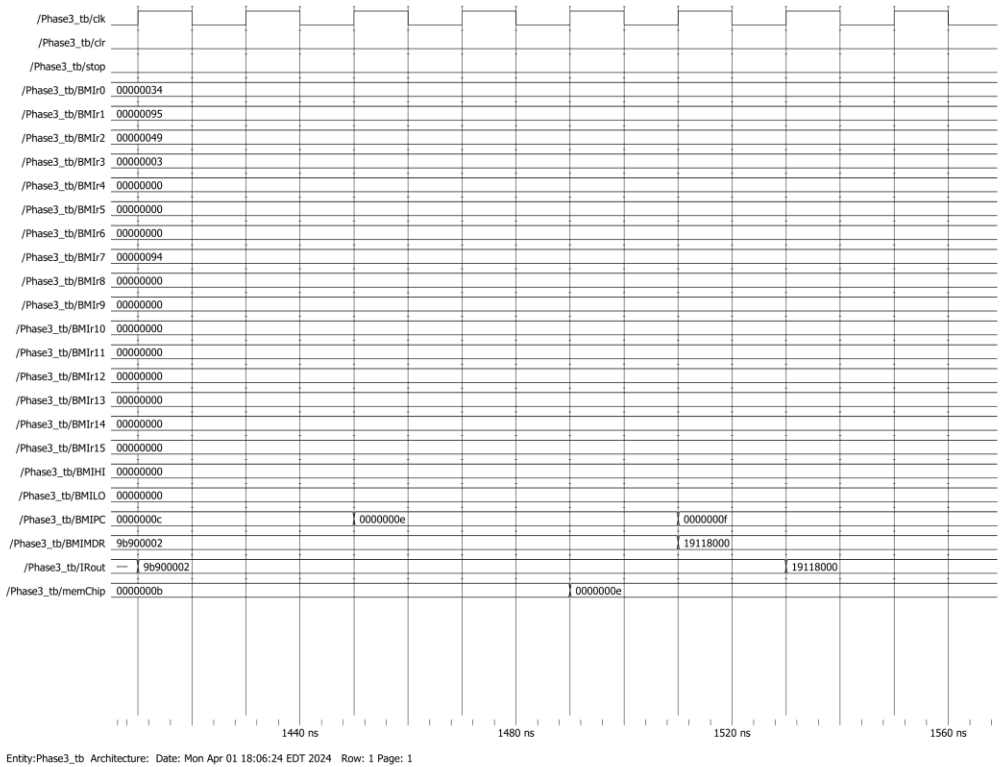


Figure 31 – brpl R7, 2; ldi R5, 4(R2); ldi R4, -3(R5);

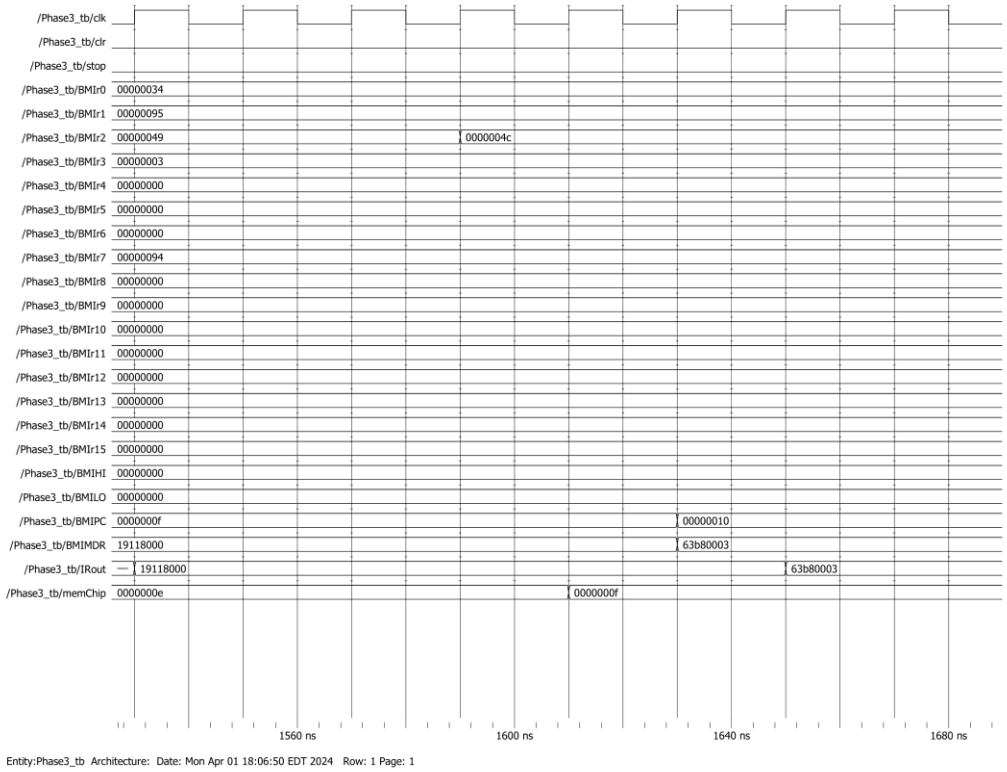


Figure 32 – add R2, R2, R3 ; R2 = 0x4C

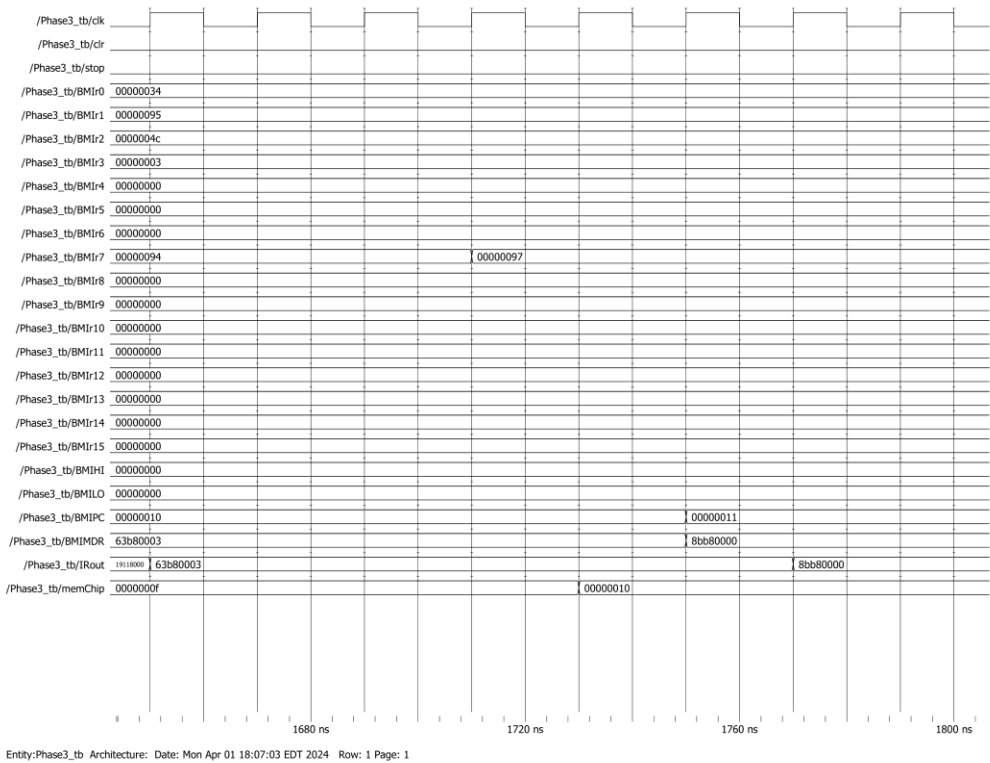


Figure 33 – addi R7, R7, 3 ; R7 = 0x97

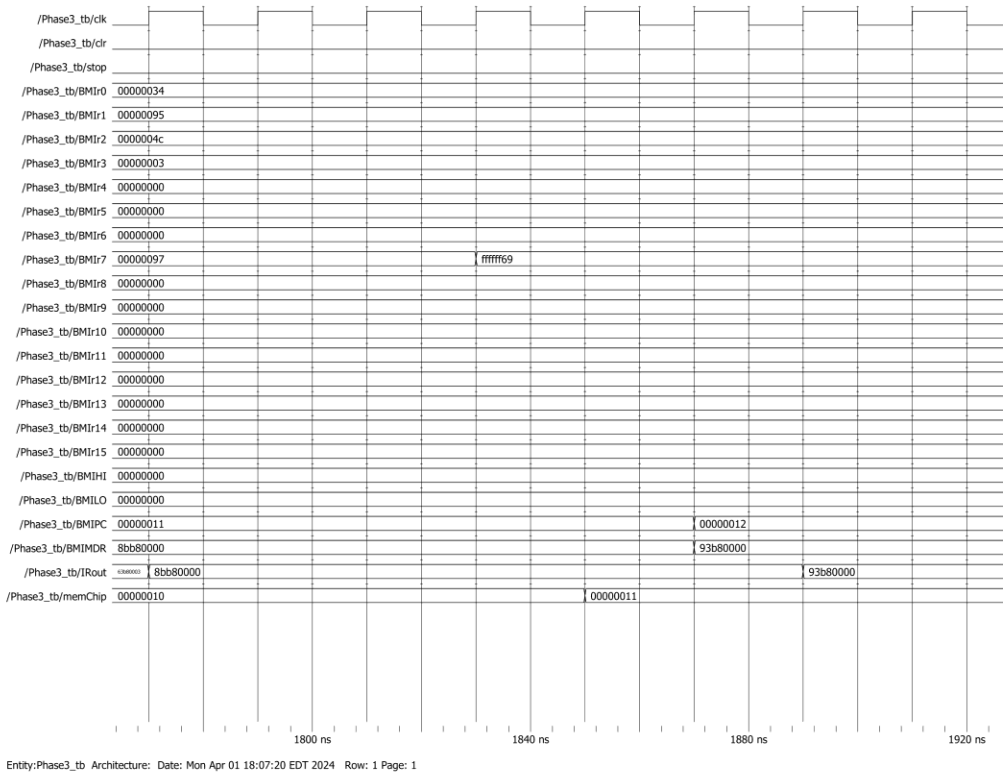


Figure 34 – neg R7, R7; R7 = 0xFFFFF69

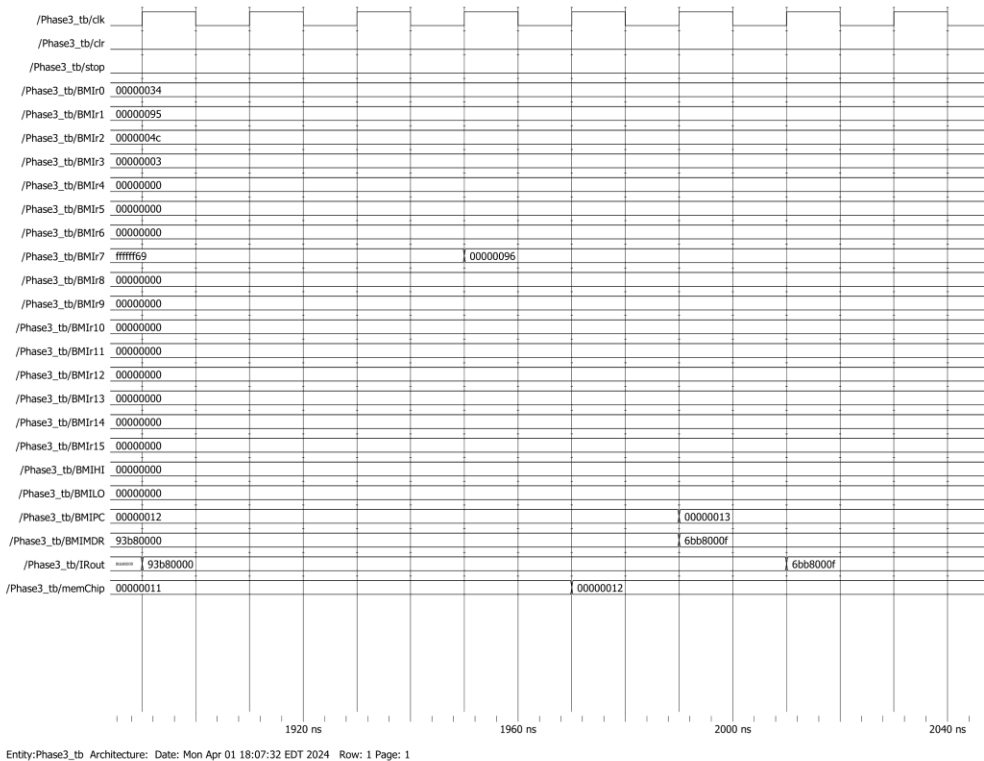


Figure 35 – not R7, R7; R7 = 0x96

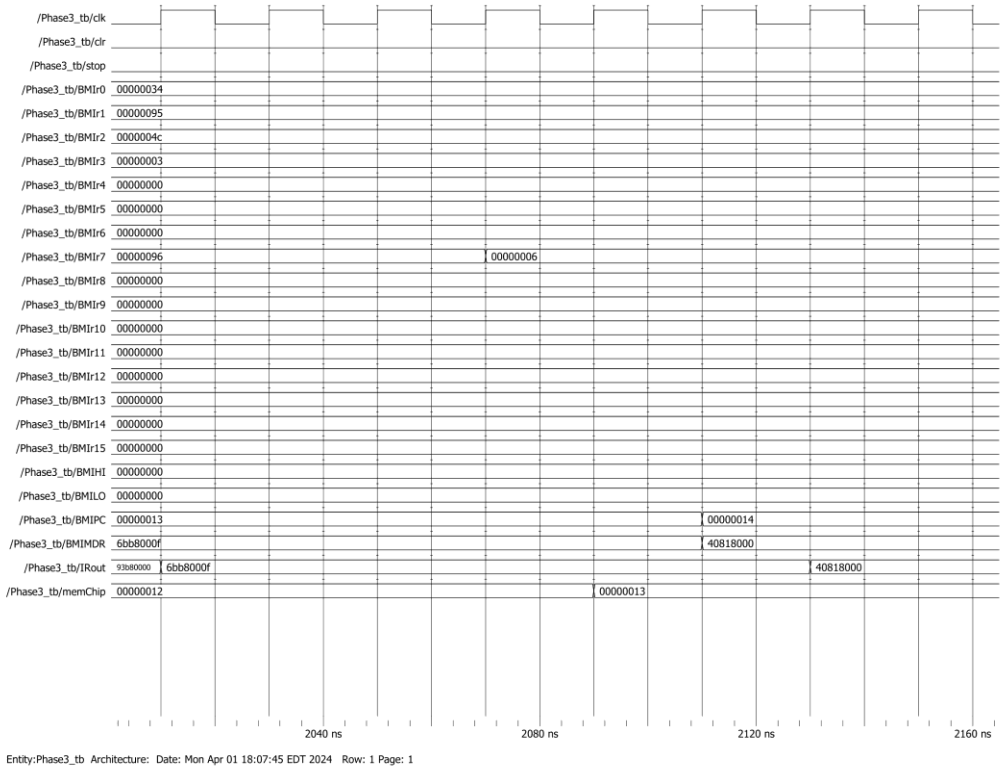


Figure 36 – andi R7, R7, 0xF ; R7 = 6

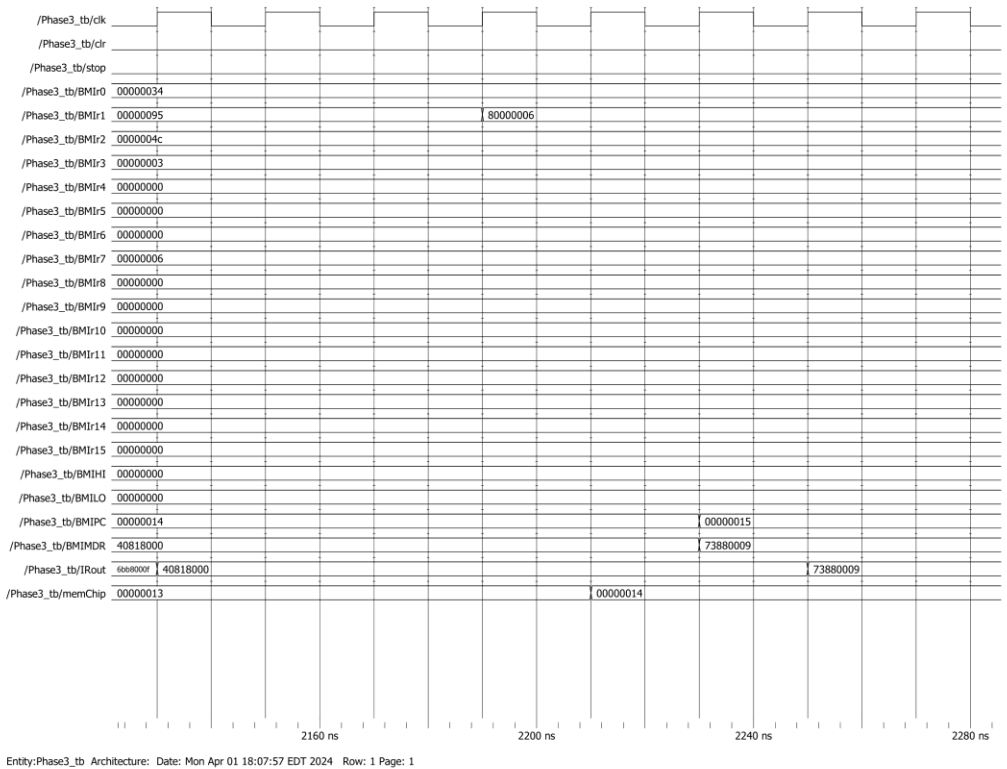


Figure 37 – ror R1, R0, R3 ; R1 = 0x80000006

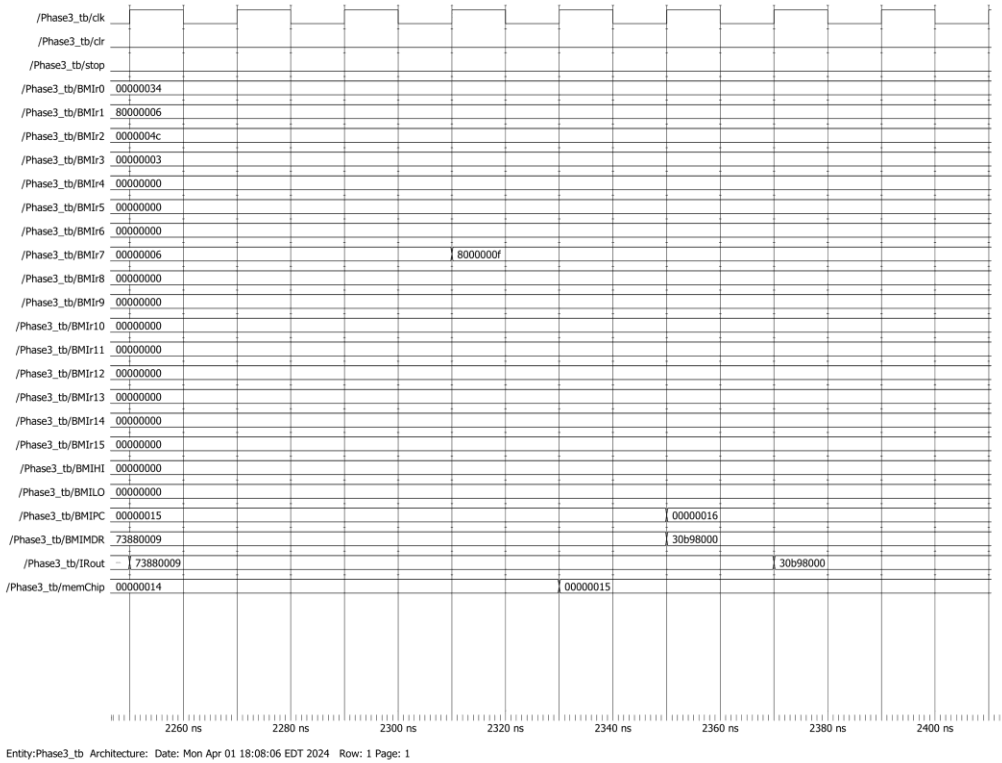


Figure 38 – ori R7, R1, 9 ; R7 = 0x8000000F

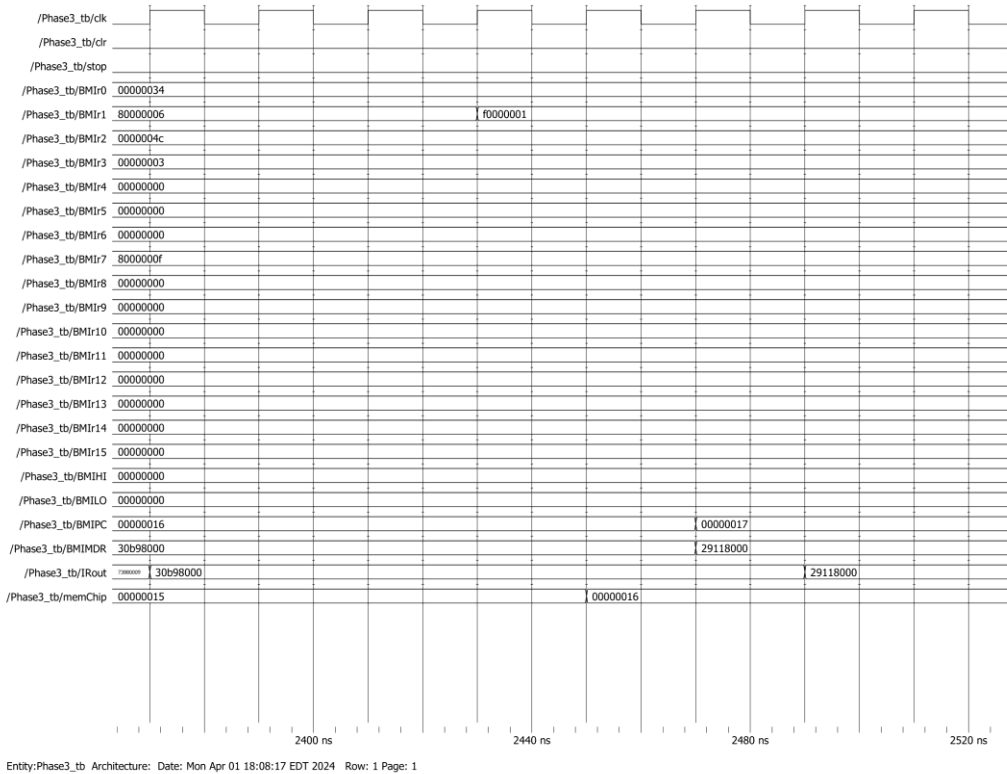


Figure 39 – shra R1, R7, R3 ; R1 = 0xF0000001

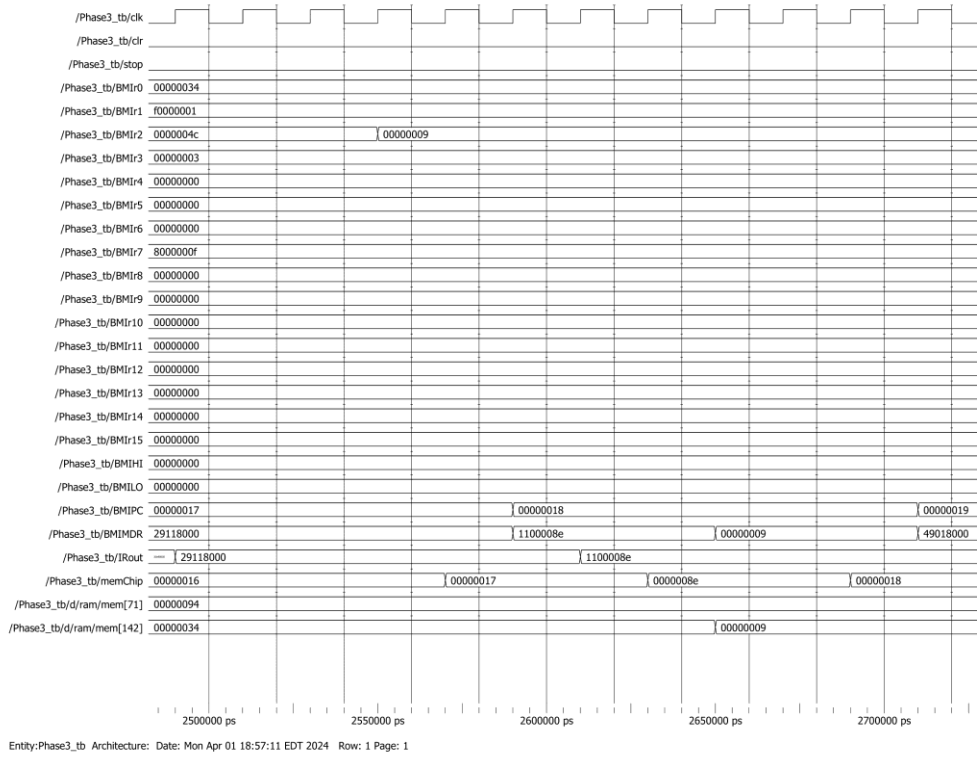


Figure 40 – shr R2, R2, R3 ; R2 = 9

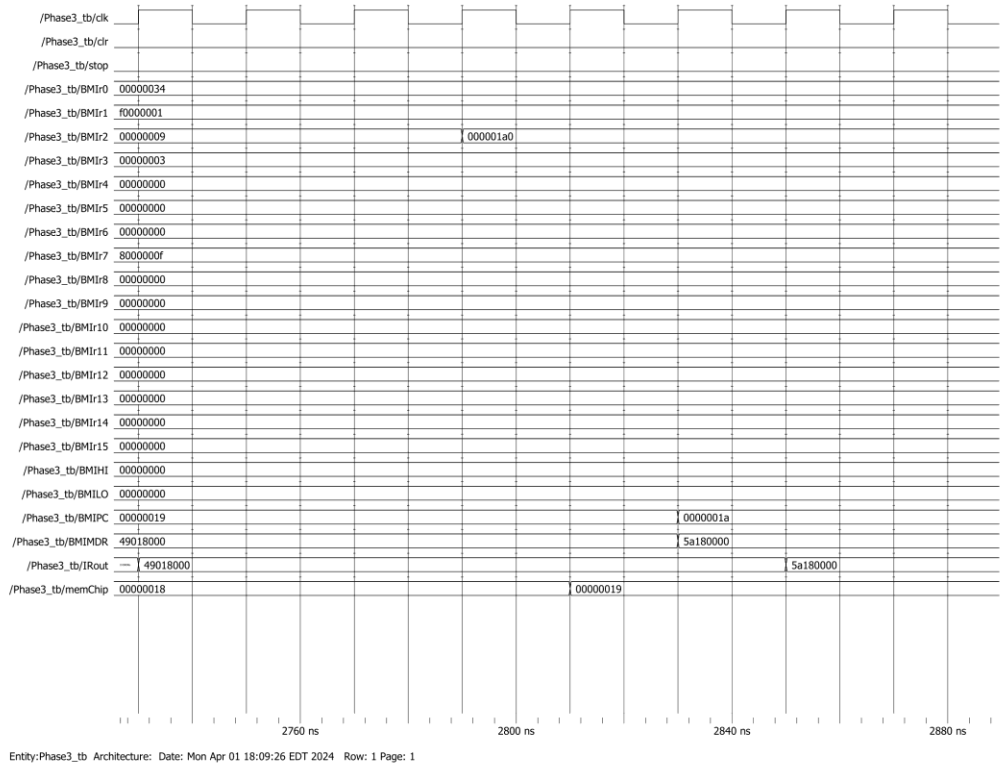


Figure 41 – st 0x8E, R2 ; (0x8E) = 9 rol R2, R0, R3 ; R2 = 0x1A0

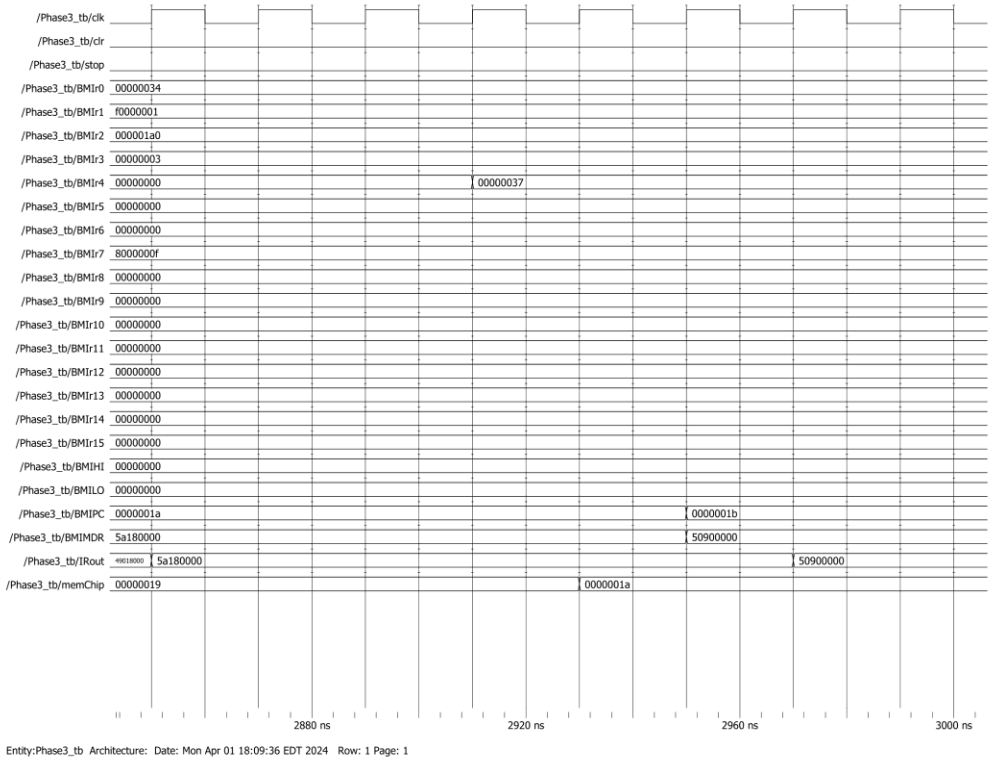


Figure 42 – or R4, R3, R0 ; R4 = 0x37

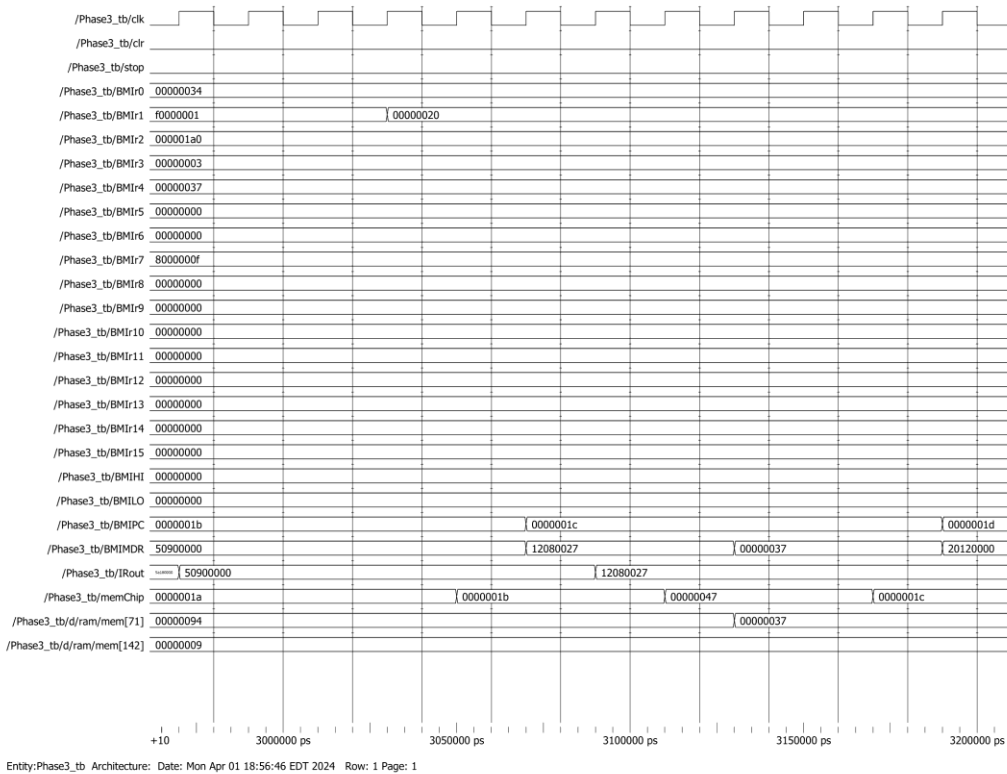


Figure 43 – and R1, R2, R0 ; R1 = 0x20

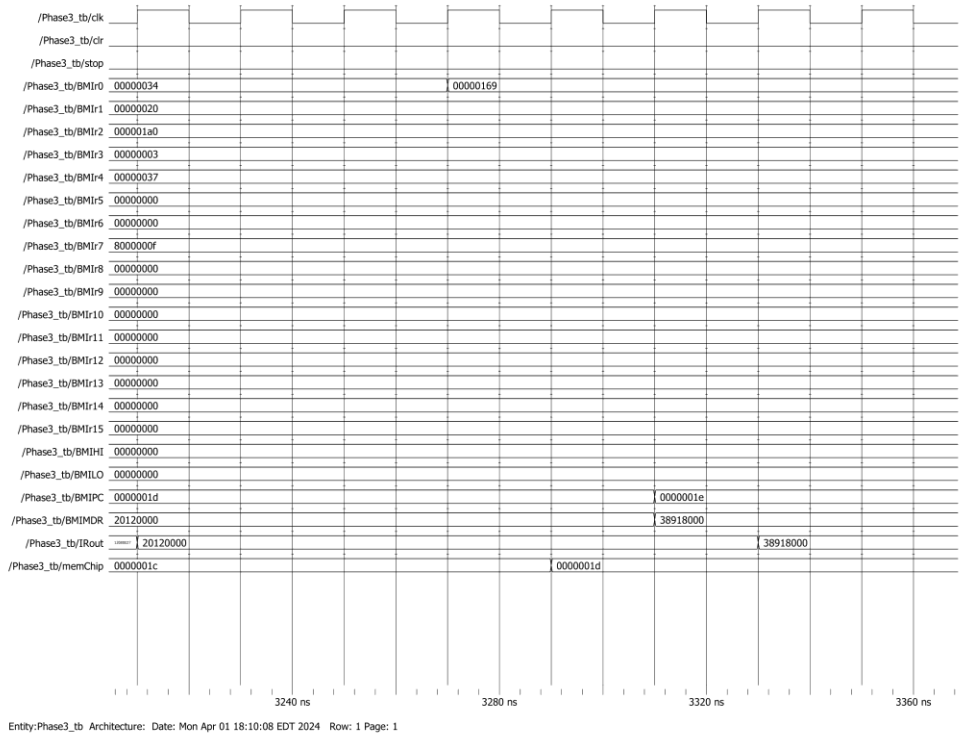


Figure 44 – *st 0x27(R1), R4 ; (0x47) = 0x37 sub R0, R2, R4 ; R0 = 0x169*

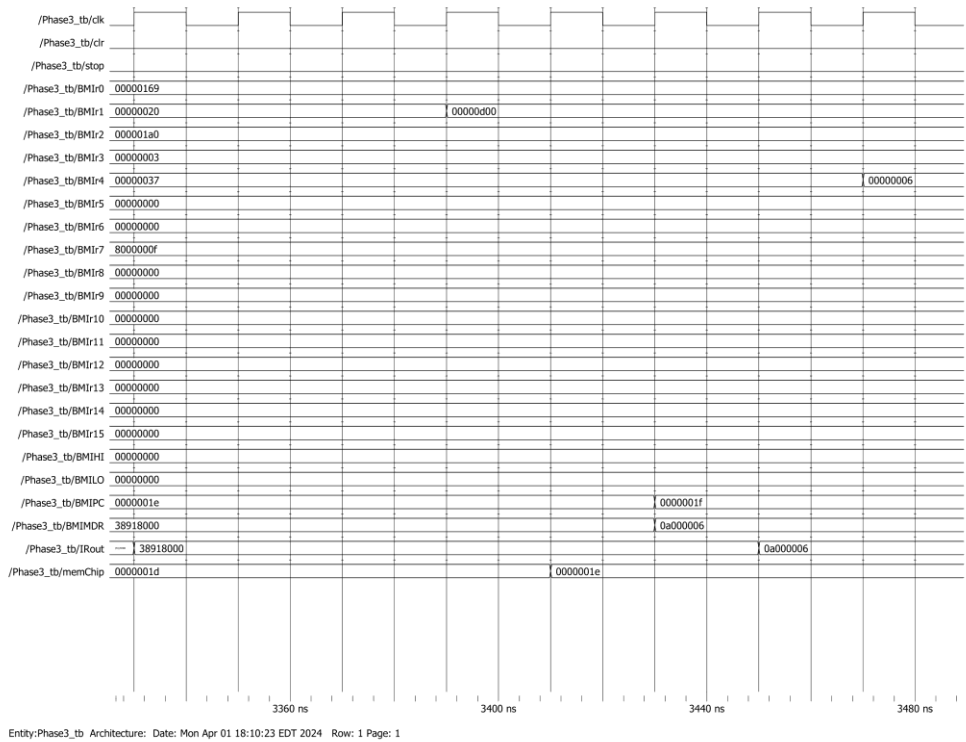
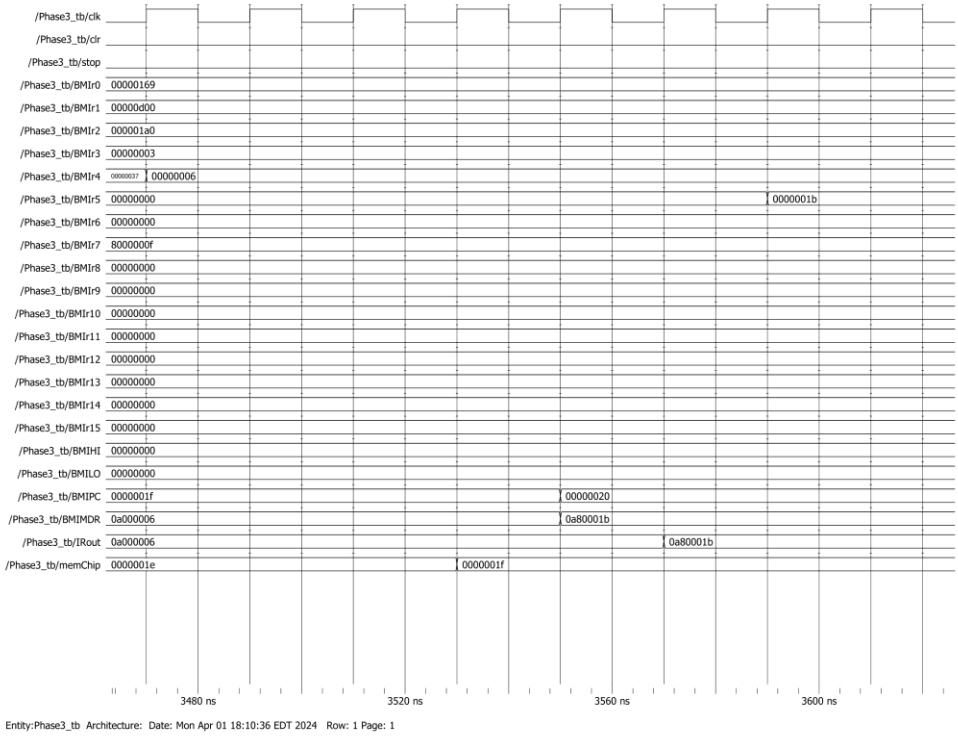
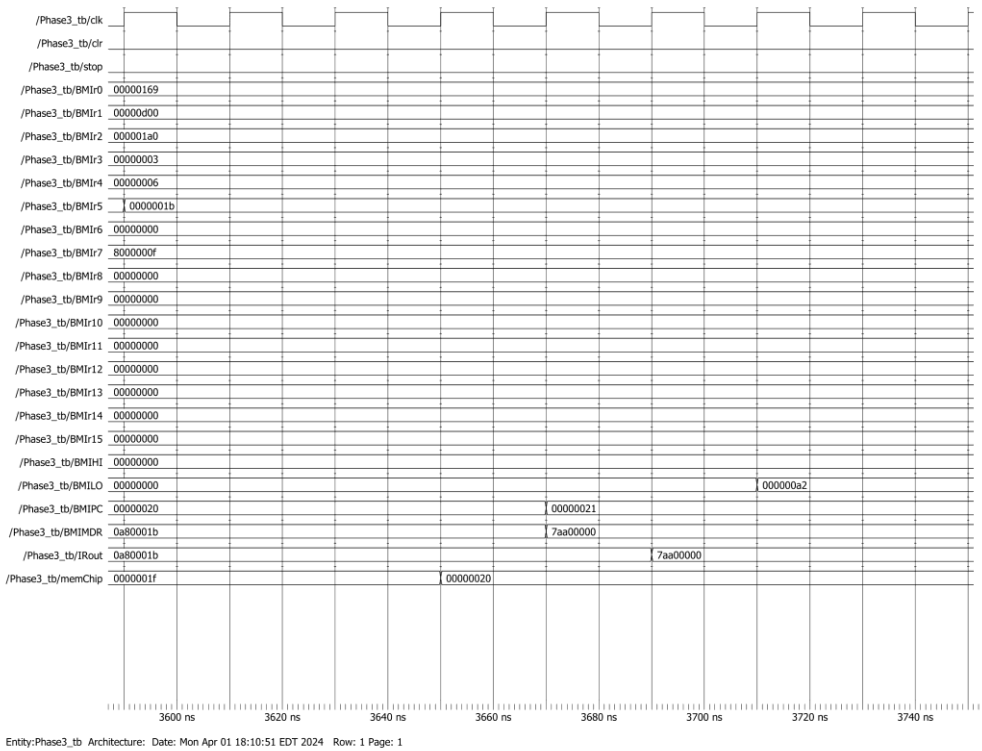


Figure 45 – *shl R1, R2, R3 ; R1 = 0xD00 ldi R4, 6 ; R4 = 6*

Figure 46 – *ldi R5, 0x1B ; R5 = 0x1B*Figure 47 – *mul R5, R4 ; HI = 0; LO = 0xA2*

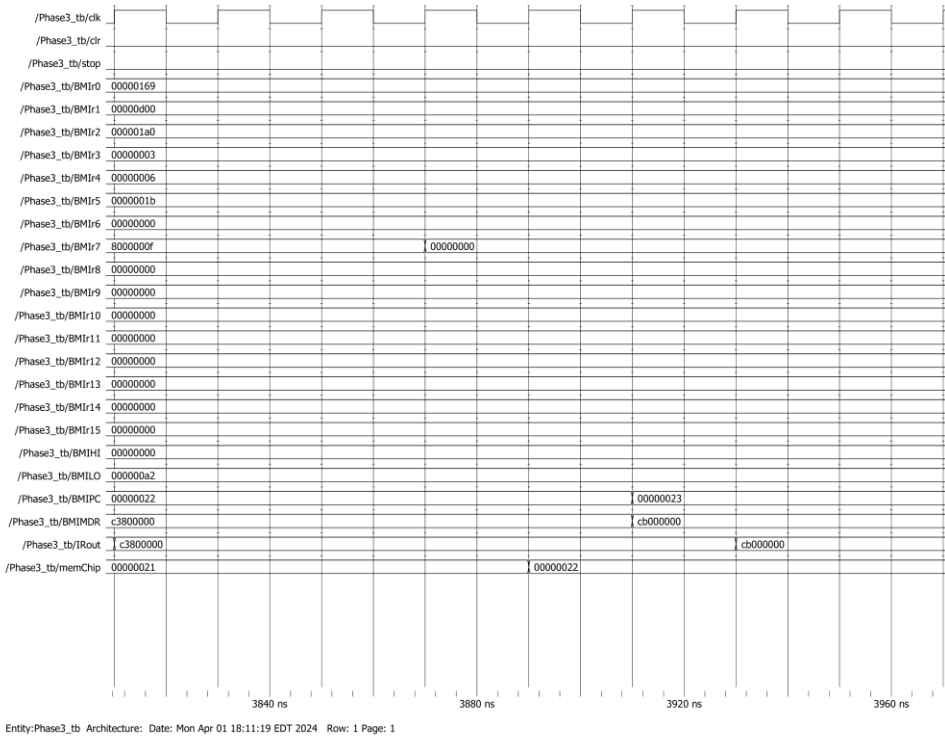


Figure 48 – mfhi R7; R7 = 0

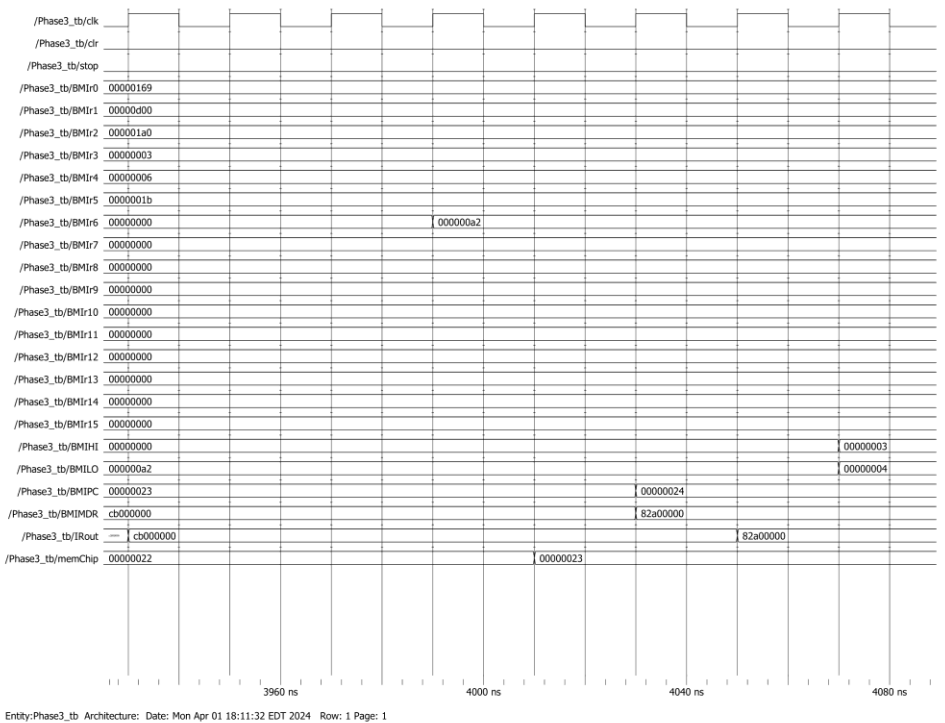


Figure 49 – mfl0 R6; R6 = 0xa2

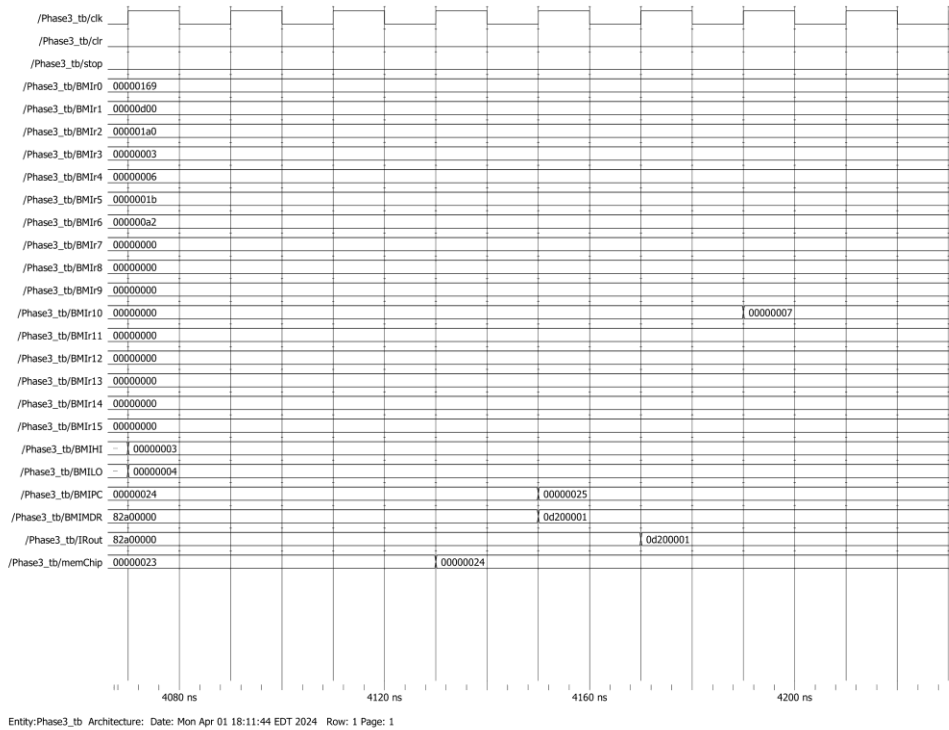


Figure 50 – *div R5, R4; HI = 3, LO = 4. Also Ldi R10, 1(R4); R10 = 7*

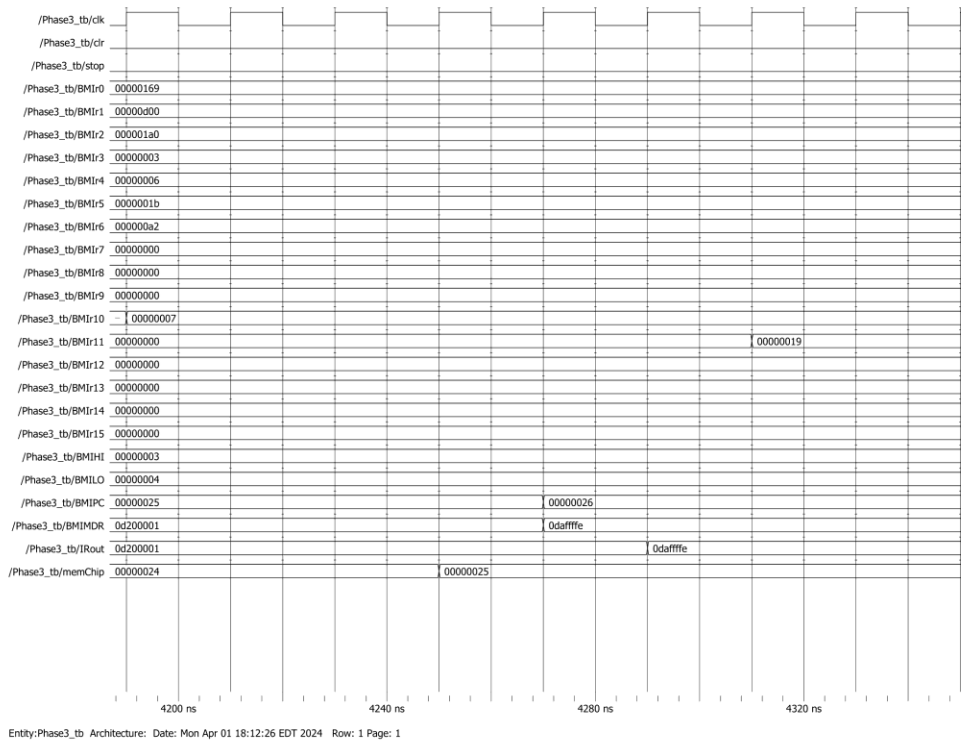


Figure 51 – *ldi R11, -2(R5); R11 = 0x19*

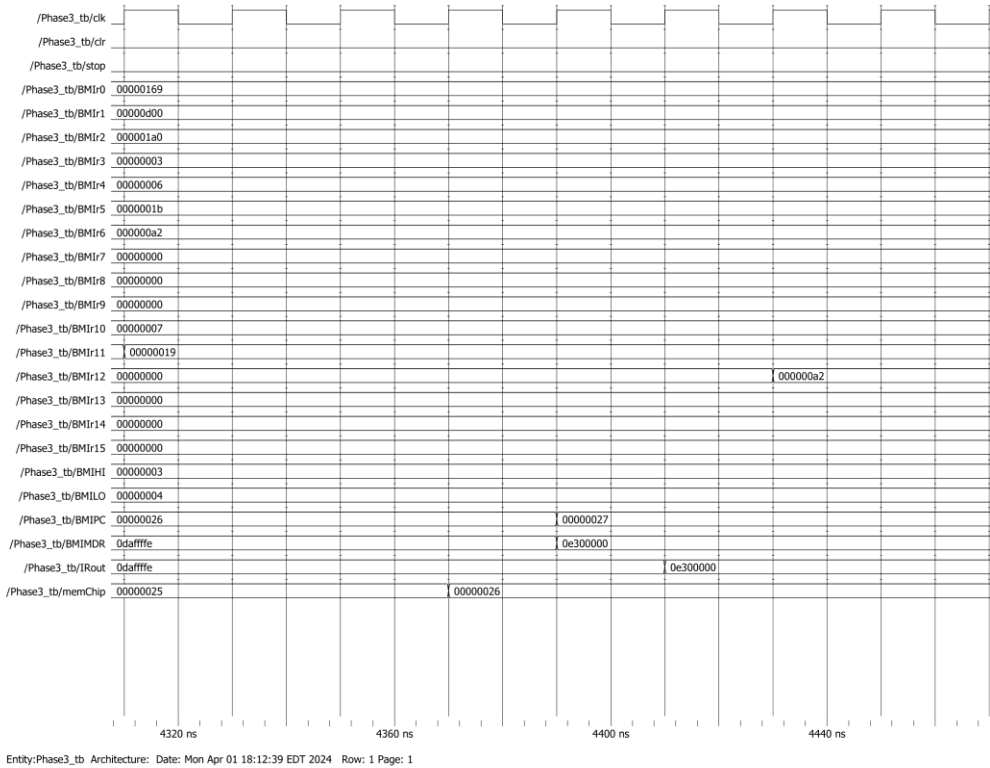


Figure 52 – *ldi R12, 0(R6); R12 = 0xa2*

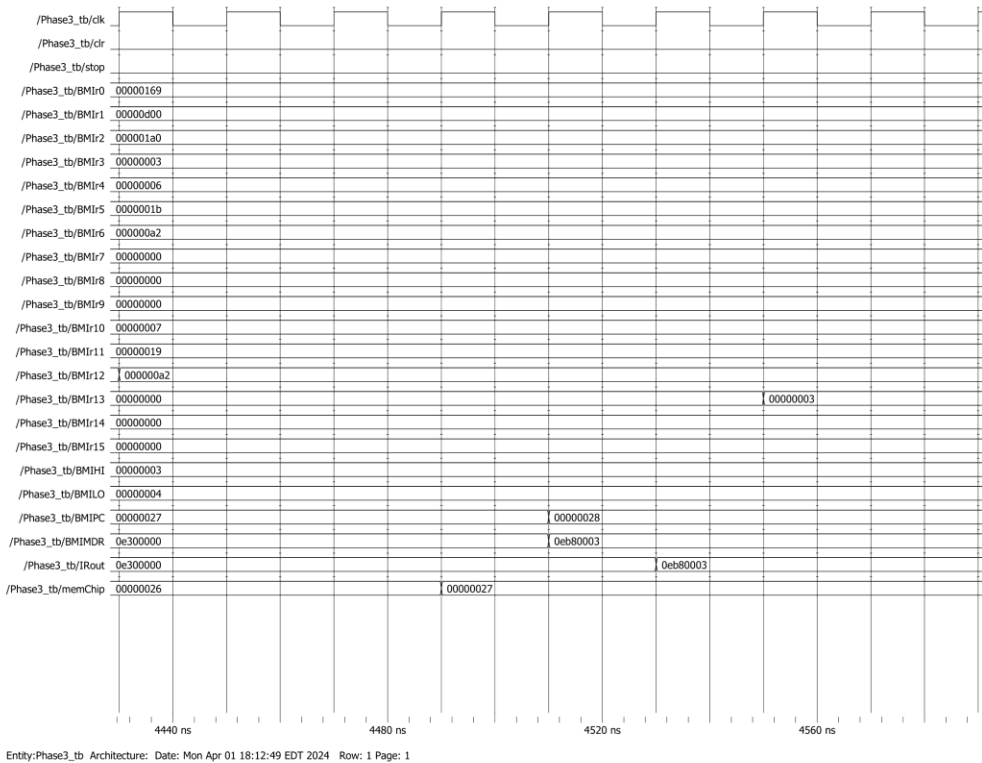


Figure 53 – *ldi R13, 3(R7); R13 = 3*

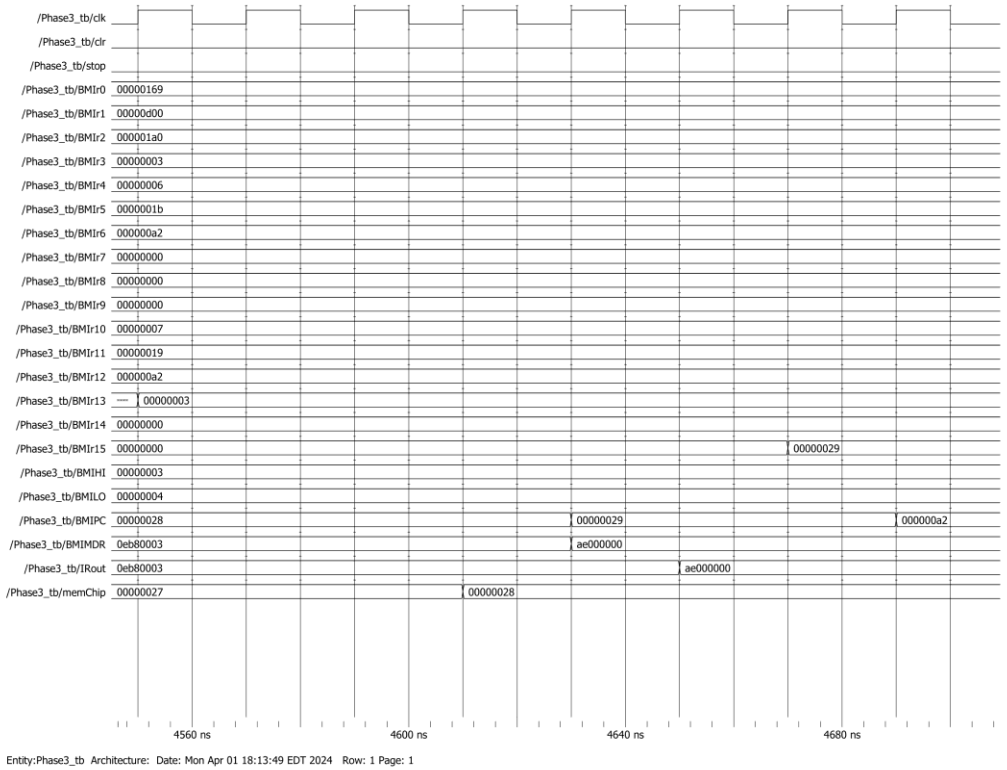


Figure 54 – jal R12

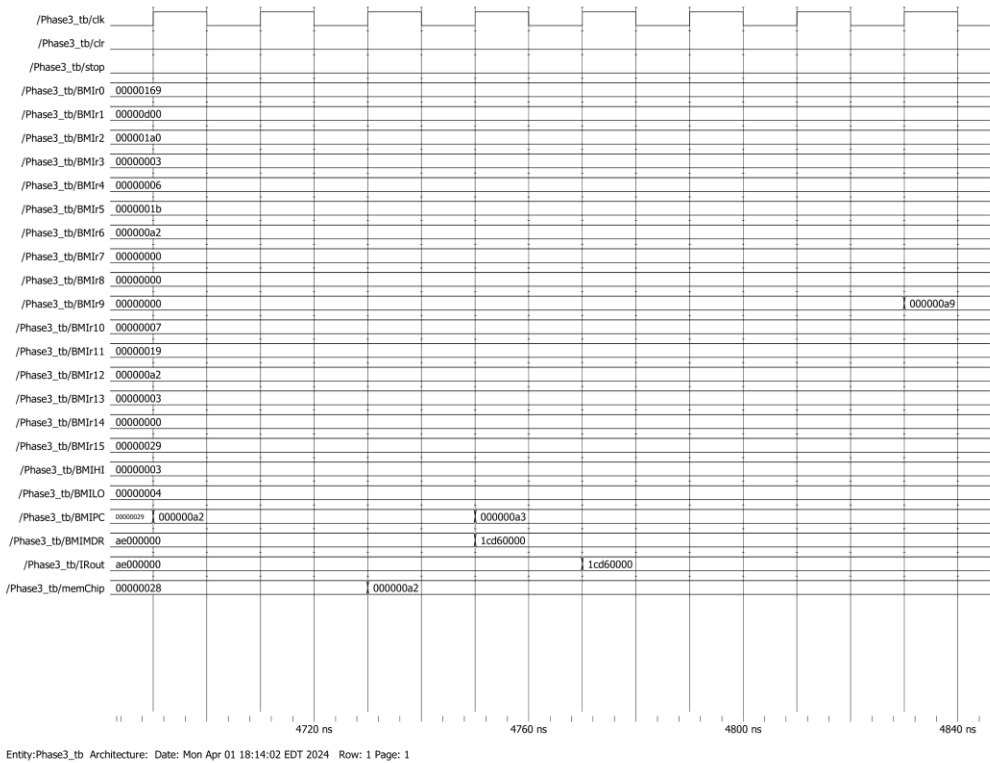


Figure 55 – halt and ORG 0xA2 and sub R8, R11, R13; R9 = 0xA9.

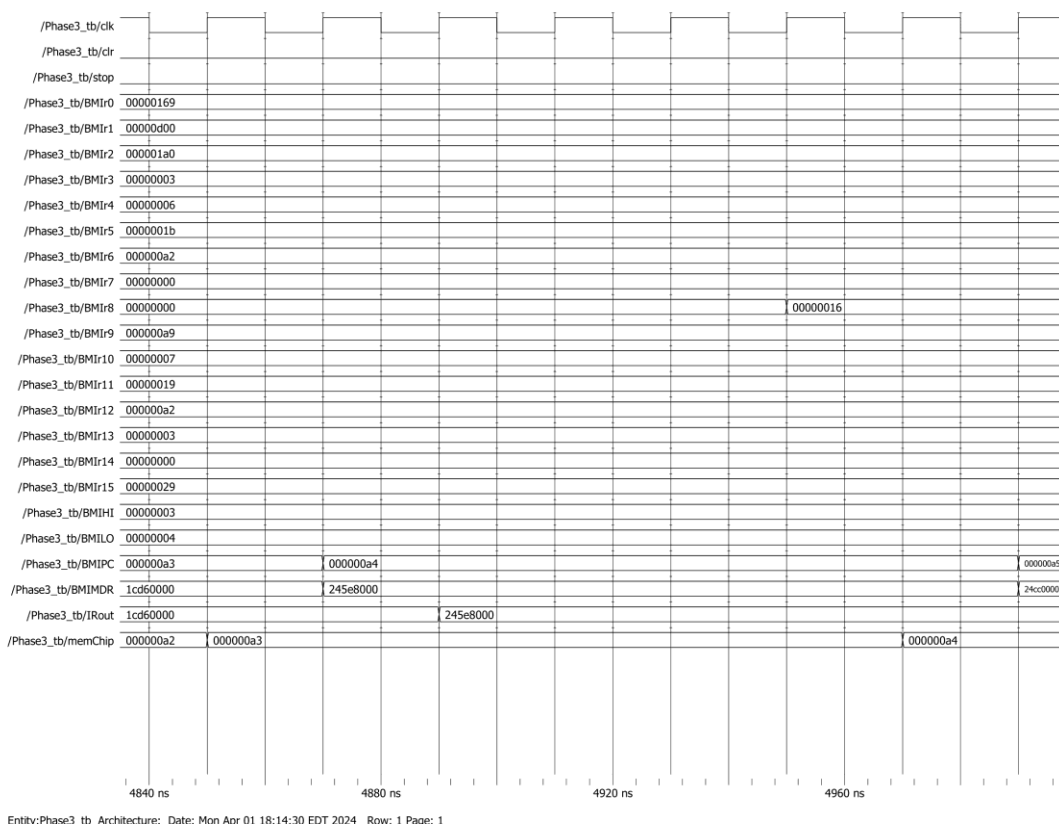


Figure 56 – sub R8, R11, R13; R8 = 0x16

Phase 3 Memory Contents

1. 09000069
2. 09100002
3. 00800047
4. 08880001
5. 000ffff9
6. 09800003
7. 09000043
8. 99180003
9. 09100006
10. 0397fffe
11. d0000000
12. 9b900002
13. 0a900004
14. 0a2ffffd
15. 19118000
16. 63b80003

17. 8bb80000
18. 93b80000
19. 6bb8000f
20. 40818000
21. 73880009
22. 30b98000
23. 29118000
24. 1100008e
25. 49018000
26. 5a180000
27. 50900000
28. 12080027
29. 20120000
30. 38918000
31. 0a000006
32. 0a80001b
33. 7aa00000
34. c3800000
35. cb000000
36. 82a00000
37. 0d200001
38. 0daffffe
39. 0e300000
40. 0eb80003
41. ae000000
42. d8000000
43. 00000000
44. 00000000
45. 00000000
46. 00000000
47. 00000000
48. 00000000
49. 00000000
50. 00000000
51. 00000000
52. 00000000
53. 00000000
54. 00000000
55. 00000000
56. 00000000
57. 00000000
58. 00000000
59. 00000000
60. 00000000

61. 00000000
62. 00000000
63. 00000000
64. 00000000
65. 00000000
66. 00000000
67. 00000000
68. 00000000
69. 00000000
70. 00000000
71. 00000000
72. 00000094
73. 00000000
74. 00000000
75. 00000000
76. 00000000
77. 00000000
78. 00000000
79. 00000000
80. 00000000
81. 00000000
82. 00000000
83. 00000000
84. 00000000
85. 00000000
86. 00000000
87. 00000000
88. 00000000
89. 00000000
90. 00000000
91. 00000000
92. 00000000
93. 00000000
94. 00000000
95. 00000000
96. 00000000
97. 00000000
98. 00000000
99. 00000000
100. 00000000
101. 00000000
102. 00000000
103. 00000000
104. 00000000

105.	00000000
106.	00000000
107.	00000000
108.	00000000
109.	00000000
110.	00000000
111.	00000000
112.	00000000
113.	00000000
114.	00000000
115.	00000000
116.	00000000
117.	00000000
118.	00000000
119.	00000000
120.	00000000
121.	00000000
122.	00000000
123.	00000000
124.	00000000
125.	00000000
126.	00000000
127.	00000000
128.	00000000
129.	00000000
130.	00000000
131.	00000000
132.	00000000
133.	00000000
134.	00000000
135.	00000000
136.	00000000
137.	00000000
138.	00000000
139.	00000000
140.	00000000
141.	00000000
142.	00000000
143.	00000034
144.	00000000
145.	00000000
146.	00000000
147.	00000000
148.	00000000

149. 00000000
150. 00000000
151. 00000000
152. 00000000
153. 00000000
154. 00000000
155. 00000000
156. 00000000
157. 00000000
158. 00000000
159. 00000000
160. 00000000
161. 00000000
162. 00000000
163. 1cd60000
164. 245e8000
165. 24cc0000
166. a7800000

Phase 4 Memory Contents

1. 09000069
2. 09100002
3. 00800047
4. 08880001
5. 000ffff9
6. 09800003
7. 09000043
8. 99180003
9. 09100006
10. 0397fffe
11. d0000000
12. 9b900002
13. 0a900004
14. 0a2ffffd
15. 19118000
16. 63b80003
17. 8bb80000
18. 93b80000
19. 6bb8000f
20. 40818000
21. 73880009
22. 30b98000
23. 29118000
24. 1100008e

25. 49018000
26. 5a180000
27. 50900000
28. 12080027
29. 20120000
30. 38918000
31. 0a000006
32. 0a80001b
33. 7aa00000
34. c3800000
35. cb000000
36. 82a00000
37. 0d200001
38. 0daffffe
39. 0e300000
40. 0eb80003
41. ae000000
42. b2000000
43. 120000aa
44. 0880002e
45. 0b800001
46. 0a800028
47. ba000000
48. 0aafffff
49. 9a800008
50. 030000f0
51. 0b37ffff
52. d0000000
53. 9b0ffffd
54. 2a238000
55. 9a0ffff7
56. 020000aa
57. a0800000
58. 0a000055
59. ba000000
60. d8000000
61. 00000000
62. 00000000
63. 00000000
64. 00000000
65. 00000000
66. 00000000
67. 00000000
68. 00000000

69. 00000000
70. 00000000
71. 00000000
72. 00000094
73. 00000000
74. 00000000
75. 00000000
76. 00000000
77. 00000000
78. 00000000
79. 00000000
80. 00000000
81. 00000000
82. 00000000
83. 00000000
84. 00000000
85. 00000000
86. 00000000
87. 00000000
88. 00000000
89. 00000000
90. 00000000
91. 00000000
92. 00000000
93. 00000000
94. 00000000
95. 00000000
96. 00000000
97. 00000000
98. 00000000
99. 00000000
100. 00000000
101. 00000000
102. 00000000
103. 00000000
104. 00000000
105. 00000000
106. 00000000
107. 00000000
108. 00000000
109. 00000000
110. 00000000
111. 00000000
112. 00000000

113.	00000000
114.	00000000
115.	00000000
116.	00000000
117.	00000000
118.	00000000
119.	00000000
120.	00000000
121.	00000000
122.	00000000
123.	00000000
124.	00000000
125.	00000000
126.	00000000
127.	00000000
128.	00000000
129.	00000000
130.	00000000
131.	00000000
132.	00000000
133.	00000000
134.	00000000
135.	00000000
136.	00000000
137.	00000000
138.	00000000
139.	00000000
140.	00000000
141.	00000000
142.	00000000
143.	00000034
144.	00000000
145.	00000000
146.	00000000
147.	00000000
148.	00000000
149.	00000000
150.	00000000
151.	00000000
152.	00000000
153.	00000000
154.	00000000
155.	00000000
156.	00000000

157.	00000000
158.	00000000
159.	00000000
160.	00000000
161.	00000000
162.	00000000
163.	1cd60000
164.	245e8000
165.	24cc0000
166.	a7800000
167.	00000000
168.	00000000
169.	00000000
170.	00000000
171.	00000000
172.	00000000
173.	00000000
174.	00000000
175.	00000000
176.	00000000
177.	00000000
178.	00000000
179.	00000000
180.	00000000
181.	00000000
182.	00000000
183.	00000000
184.	00000000
185.	00000000
186.	00000000
187.	00000000
188.	00000000
189.	00000000
190.	00000000
191.	00000000
192.	00000000
193.	00000000
194.	00000000
195.	00000000
196.	00000000
197.	00000000
198.	00000000
199.	00000000
200.	00000000

201.	00000000
202.	00000000
203.	00000000
204.	00000000
205.	00000000
206.	00000000
207.	00000000
208.	00000000
209.	00000000
210.	00000000
211.	00000000
212.	00000000
213.	00000000
214.	00000000
215.	00000000
216.	00000000
217.	00000000
218.	00000000
219.	00000000
220.	00000000
221.	00000000
222.	00000000
223.	00000000
224.	00000000
225.	00000000
226.	00000000
227.	00000000
228.	00000000
229.	00000000
230.	00000000
231.	00000000
232.	00000000
233.	00000000
234.	00000000
235.	00000000
236.	00000000
237.	00000000
238.	00000000
239.	00000000
240.	00000000
241.	000000FF